# Advanced Computer Networks

## In-Network Computing - Hardware

Lin Wang

Fall 2022, Period 2

VU VRIJE UNIVERSITEIT AMSTERDAM

# Course outline

## Essentials

- Introduction (history, principles)
- Networking basics
- Network transport

## Data center networking

- Data center networking
- Data center transport
- Software defined networking
- Network function virtualization
- Programmable data plane
- Programmable switch architecture

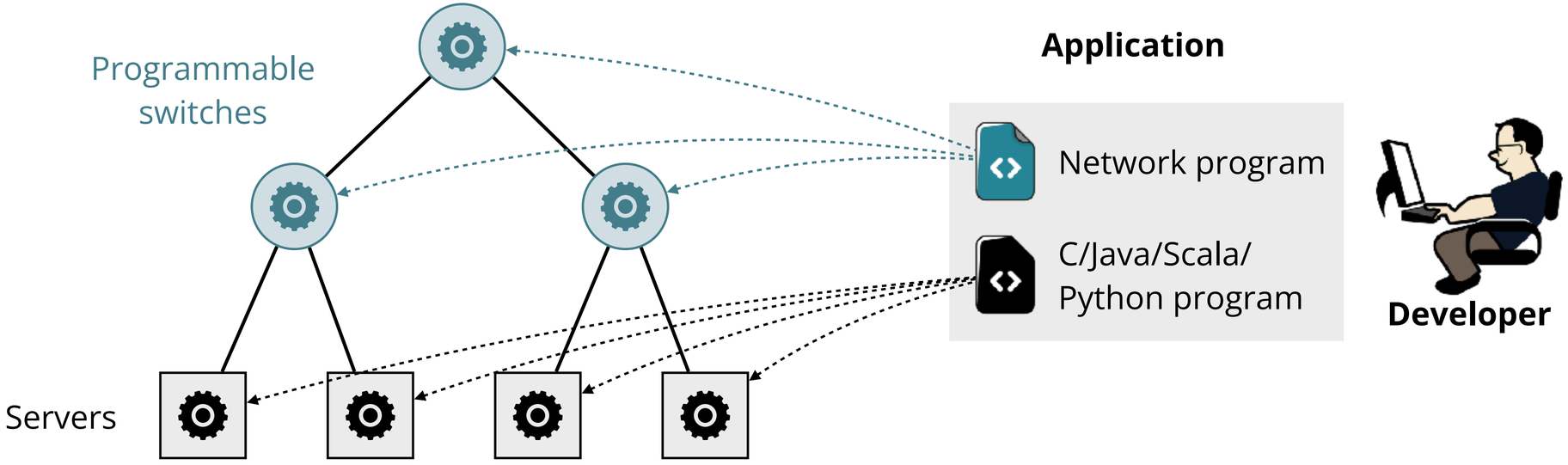
## Network innovations

- In-network computing - applications
- **In-network computing - hardware**
- Network monitoring
- ~~Machine learning for networking~~

## Guest lecture

- Fernando Ramos (University of Lisbon)

# In-network computing




How to support more in-network computing applications with **moderate modifications** to the switch architecture?

# Learning objectives

How to support **multi-dimensional table filtering** on switch data plane?

How to support **machine learning inference** on switch data plane?

# How to support multi-dimensional table filtering on switch data plane?

# Chained multi-dimensional table filtering

Important for tasks like resource-aware L4 load balancing

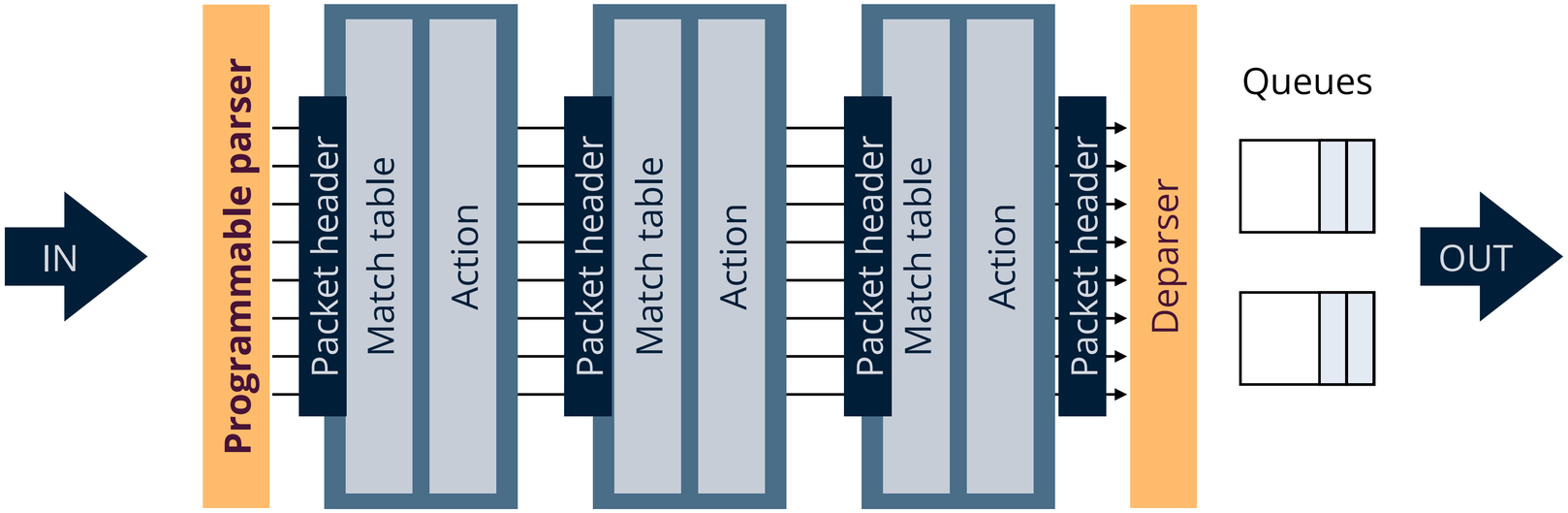
Attribute

Servers

| | Memory | Bandwidth | CPU |
|---|---|---|---|
| **Server 1** | 24GB | 800Mbps | 40% |
| **Server 2** | 10GB | 500Mbps | 10% |
| **Server 3** | 56Gb | 100Mbps | 90% |
| **Server 4** | 32GB | 450Mbps | 30% |

Find a server with **available memory** over 20GB and **available bandwidth** over 400Mbps.
From the filtered set, choose server with **least CPU utilization**.

# Current RMT-based switch architecture

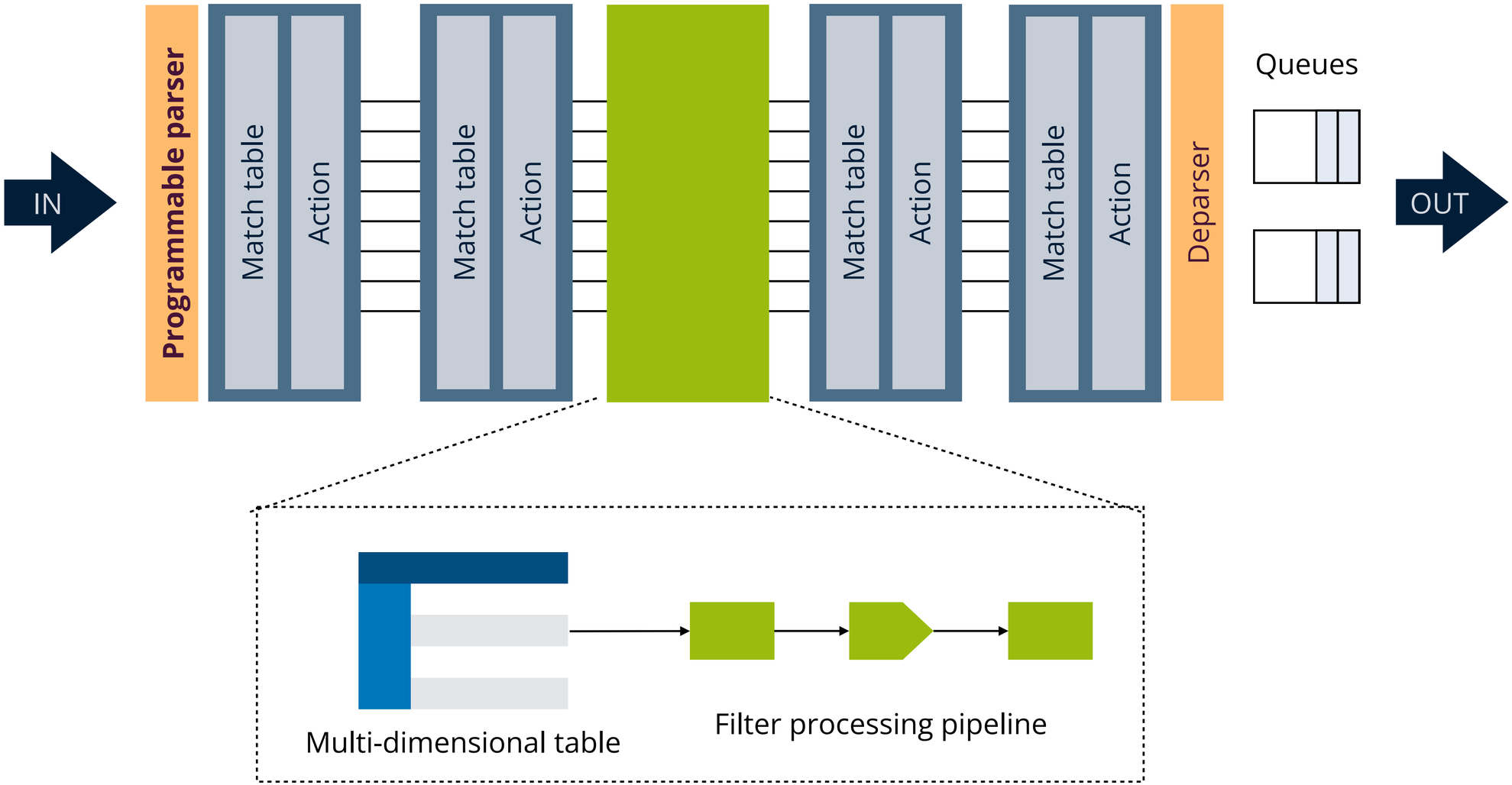



Does not support line rate multi-dimensional table filtering

**MATs:** can be used to match on attributes (in packet headers), but cannot perform custom filtering.

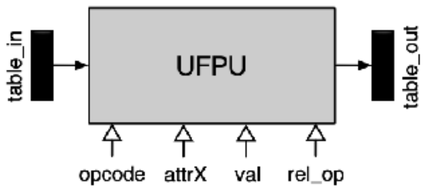
**Registers:** can be used as hash tables to apply simple stateless filtering (e.g., random selection), but requires $O(N)$ stages for a single stateful filtering.

# Thanos: programmable filtering module in the data plane



# Abstractions and primitives with UFPU

**Unary filter processing unit**

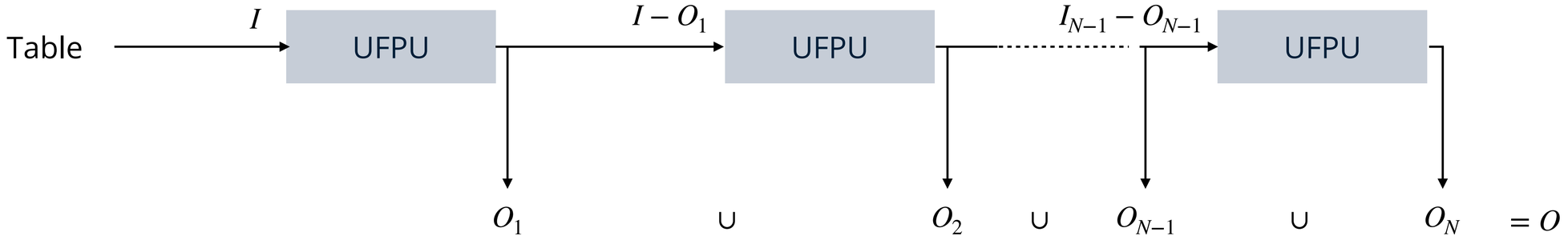


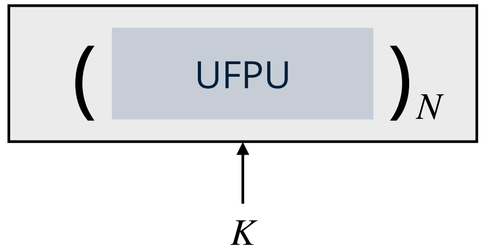

| | |
|---|---|
| `no-op(tab1)` | Bypasses all filter operations and simply copies the input tab1 to its output |
| `random(tab1)` | Outputs a table comprising a single entry from tab1 chosen uniformly at random |
| `predicate(tab1, attrX rel_op val)` | Operates on the attribute attrX of tab1, and outputs a table comprising entries whose attrX value satisfies the predicate (rel_op is relational operator) |
| `round-robin(tab1, attrX)` | Outputs a list comprising a single entry chosen cyclicly from tab1 in proportion to the entry's weight represented by attrX |
| `min/max(tab1, attrX)` | Outputs a table comprising a single entry from tab1 whose attrX value is the lowest/highest amongst all the entries in tab1 |

# Chaining UFPUs

Filter $N$ least CPU utilized load balancing servers

Filter $N$ random load balancing servers



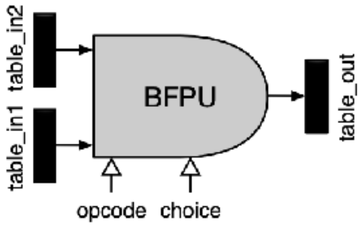




$K$-UFPU: as the basic computing units instead of UFPU

Specifies the length of the chain, max $N$

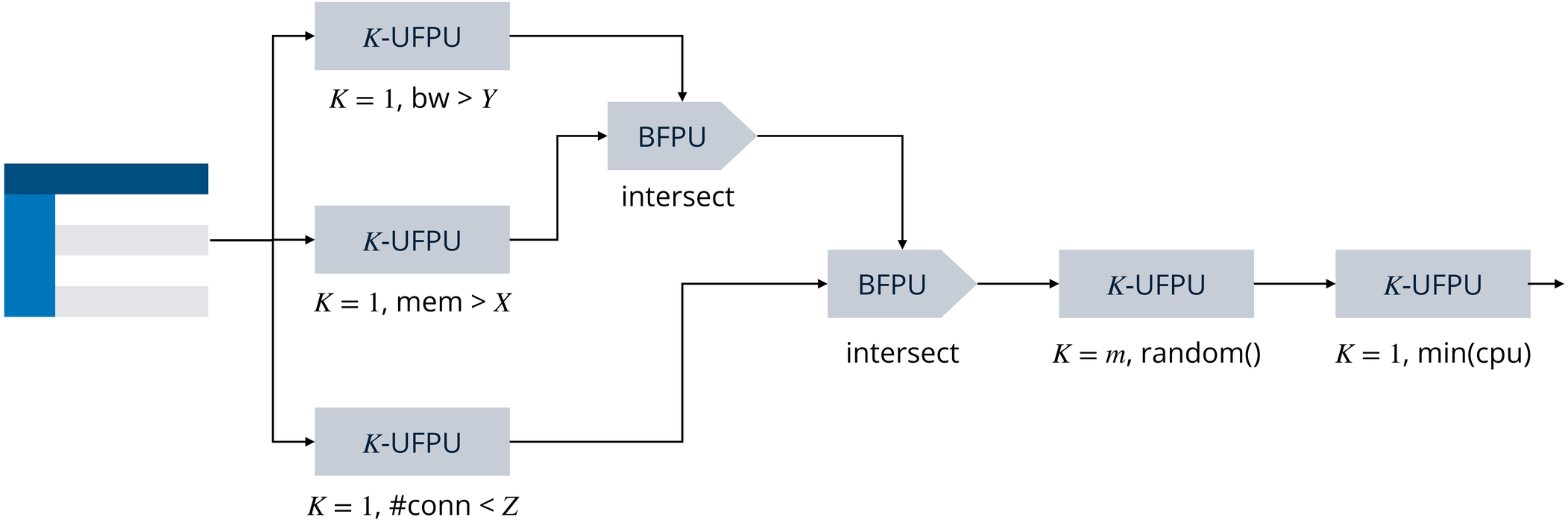
# Abstractions and primitives with BFPUs

**Binary filter processing unit**




| | |
|---|---|
| `no-op(tab1, tab2, choice)` | Bypasses all operation and outputs the table specified by the choice |
| `union(tab1, tab2)` | Outputs the table with all the entries from both tab1 and tab2 |
| `intersection(tab1, tab2)` | Outputs the table with entries common to both tab1 and tab2 |
| `difference(tab1, tab2)` | Outputs the table with entries in tab1 but not in tab2 |

# Example serial chain filter pipeline

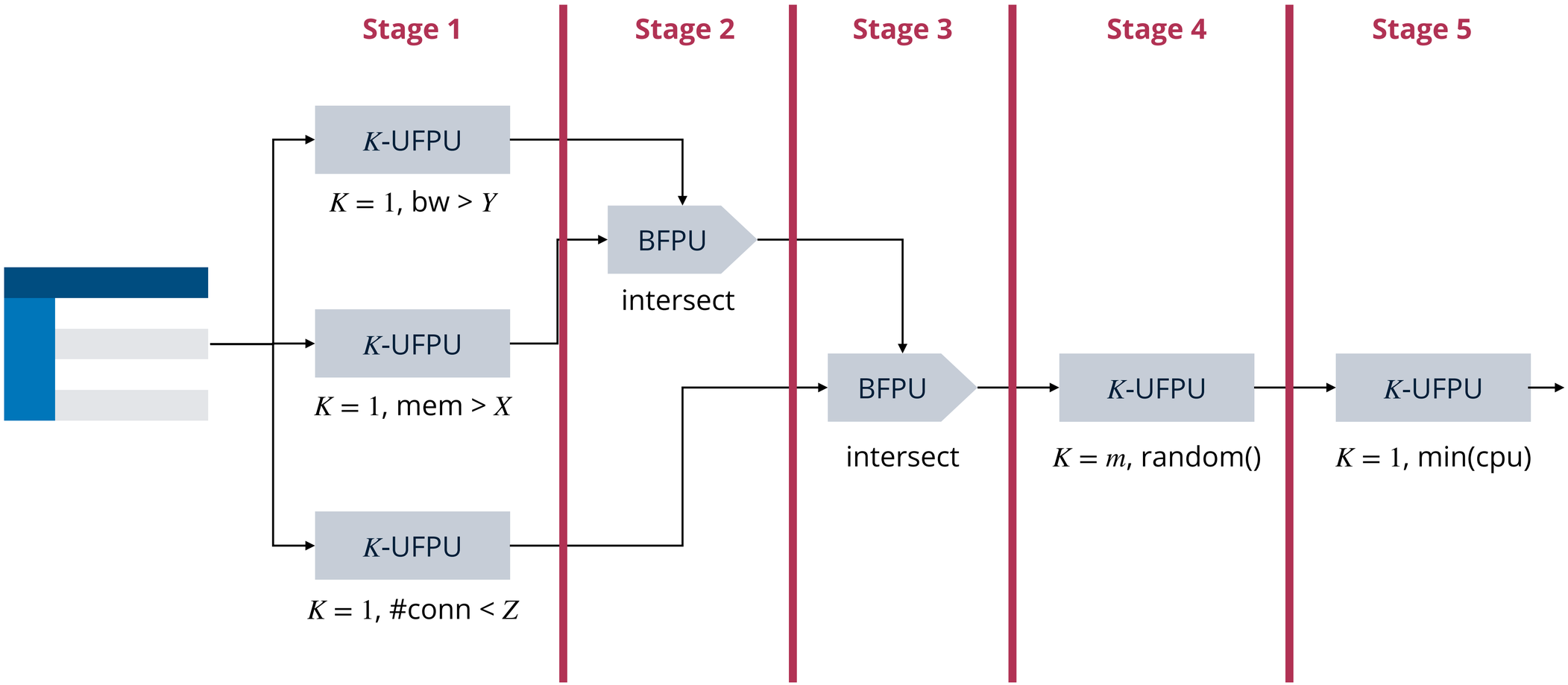



Filter all load balancing servers with available memory > $X$ **and** available bandwidth > $Y$ **and** #conn < $Z$

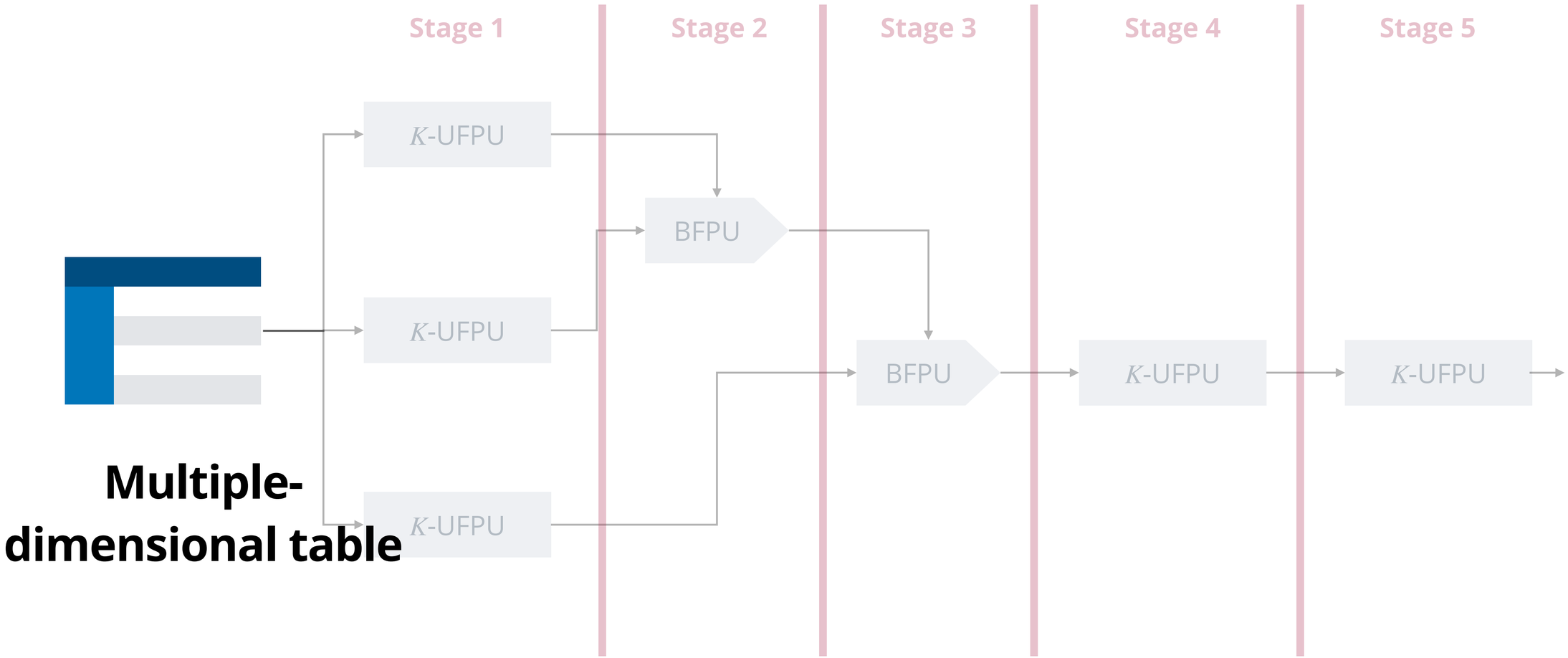
Filter $m$ load balancing servers at random

Filter load balancing server with minimal CPU utilization

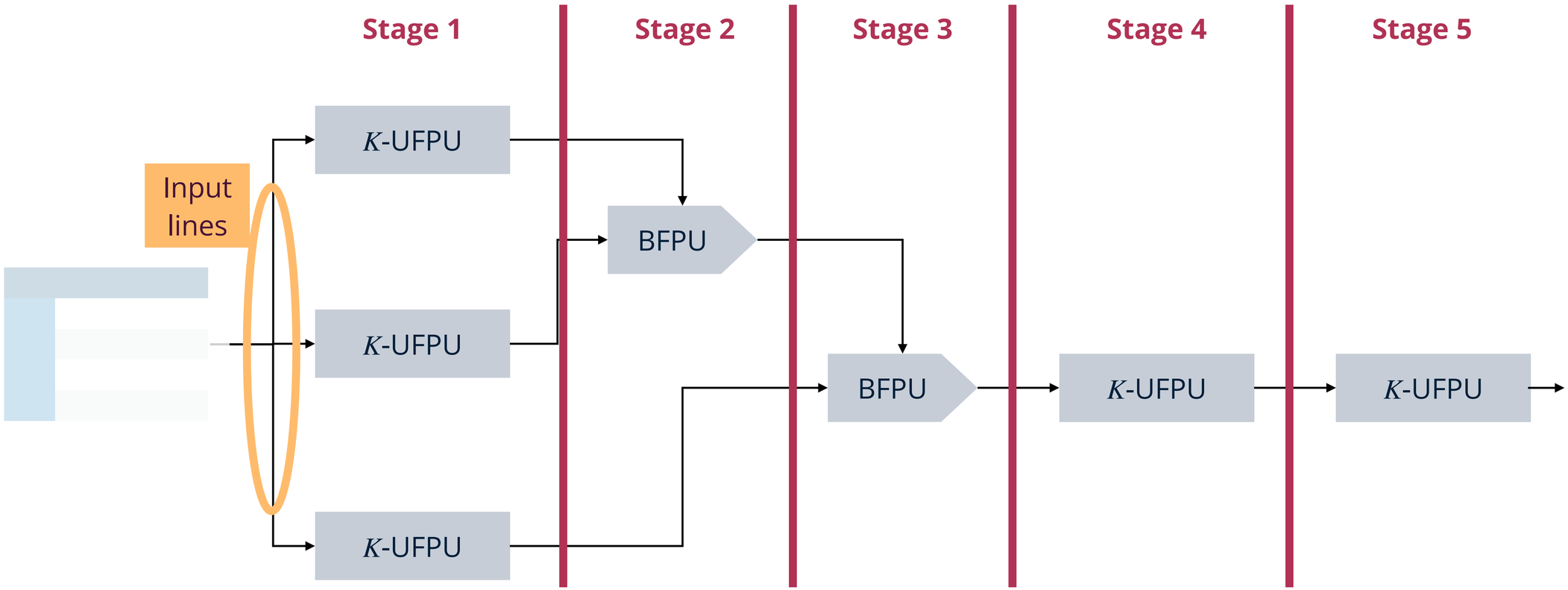
# Example serial chain filter pipeline



# Hardware design



# Hardware design




**Programmable filter pipeline**

# Multi-dimensional relational table

Attribute

Server

| | Memory | Bandwidth | CPU |
|---|---|---|---|
| **Server 1** | 24GB | 800Mbps | 40% |
| **Server 2** | 10GB | 500Mbps | 10% |
| **Server 3** | 56Gb | 100Mbps | 90% |
| **Server 4** | 32GB | 450Mbps | 30% |

How to design an **efficient data structure** that allows line-rate read, write, and update?!

# Limitations of classic data structures

## No universal data structures

- Range trees / B-trees for range filtering
- Heap for min/max filtering
- Disjoint set data structure for set operations
- Either compromise on **performance** of certain operations (since they are customized for certain operations) or pay the **space cost** of maintaining multiple data structures over the same data

## Hierarchical structure

- Fundamental $O(\log N)$ latency
- Hard to pipeline [1]

[1] Yi-Hua E. Yang and Viktor K. Prasanna. “High Throughput and Large Capacity Pipelined Dynamic Search Tree on FPGA”. FPGA, 2010.

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as

| ID | X | Y |
|---|---|---|
| 1 | 4 | 6 |
| 2 | 15 | 8 |
| 3 | 15 | 19 |
| 4 | 22 | 19 |

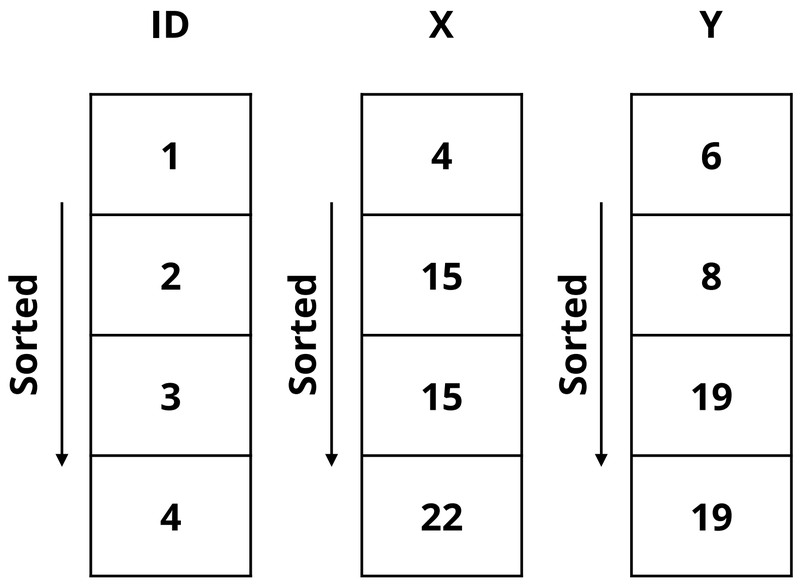
Store each dimension as **flat** list of flip-flops
→ allows parallel access and processing

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as



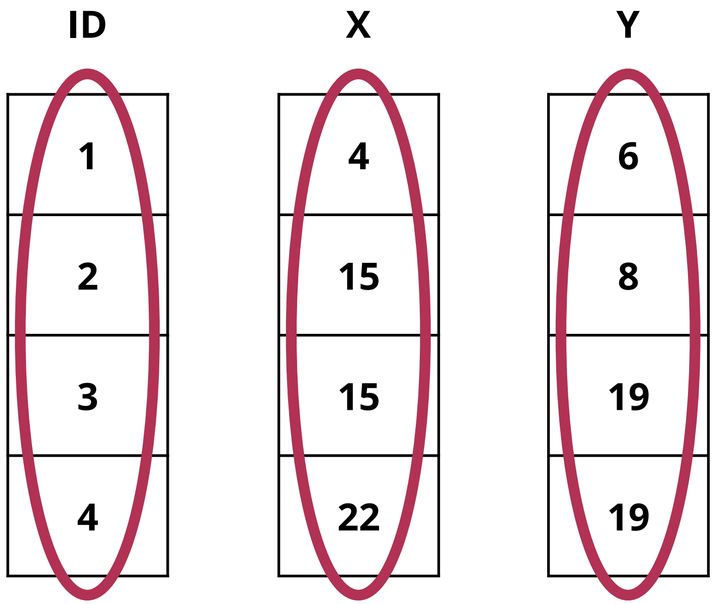

Each list is kept sorted → allows fast min/max filter operations

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as



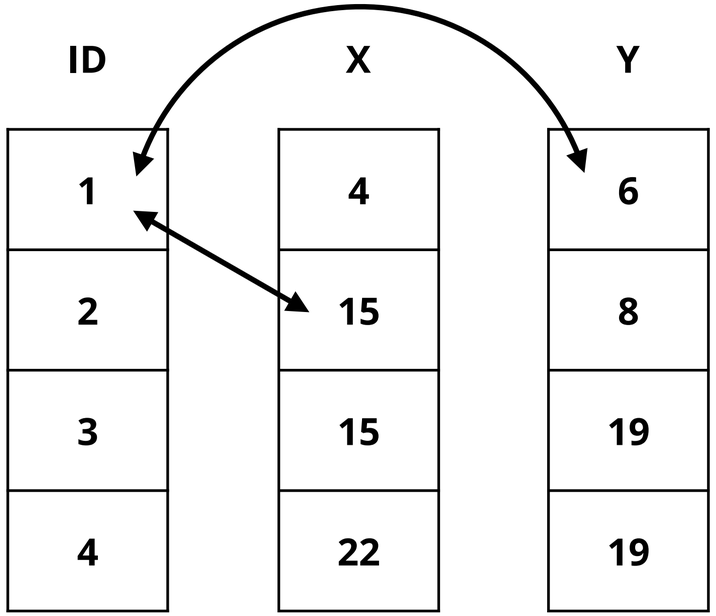

Each dimension is stored as an **independent** list
→ allows parallel filter operations on multiple dimensions

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as



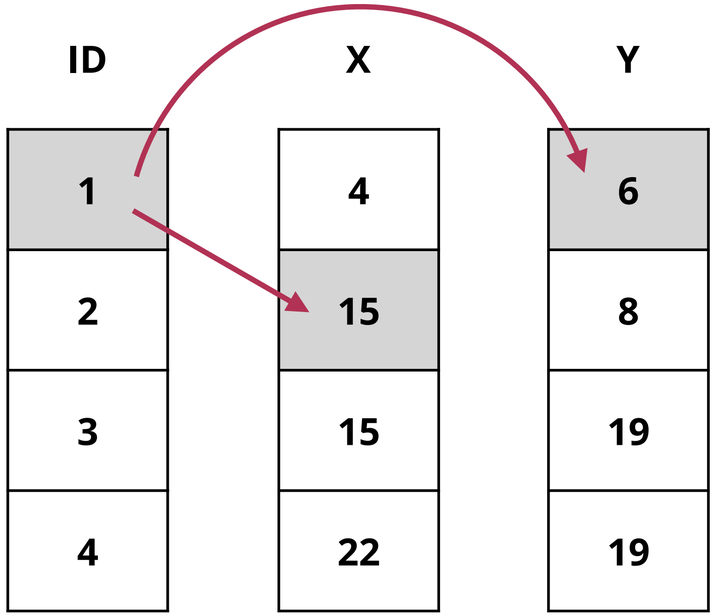

Bidirectional mapping between ID and attributes

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as




Bidirectional mapping between ID and attributes

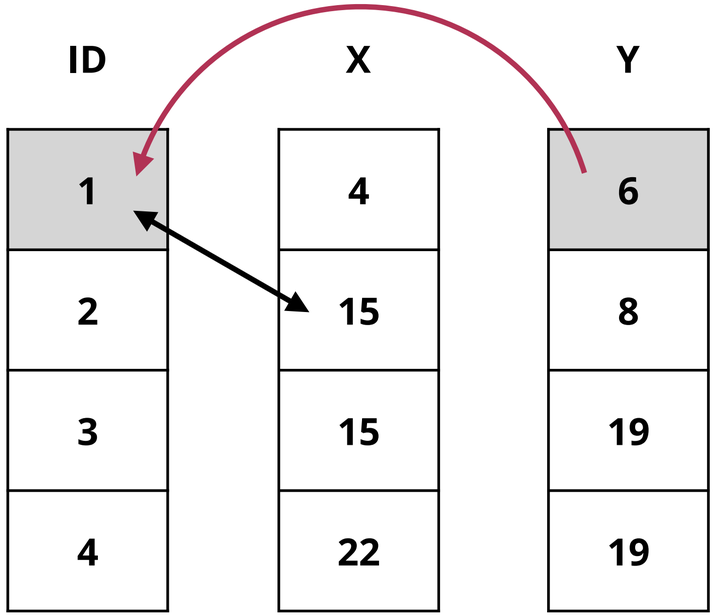
Forward map keeps track of which attributes belong to which ID

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as




Bidirectional mapping between ID and attributes

Forward map keeps track of which attributes belong to which ID

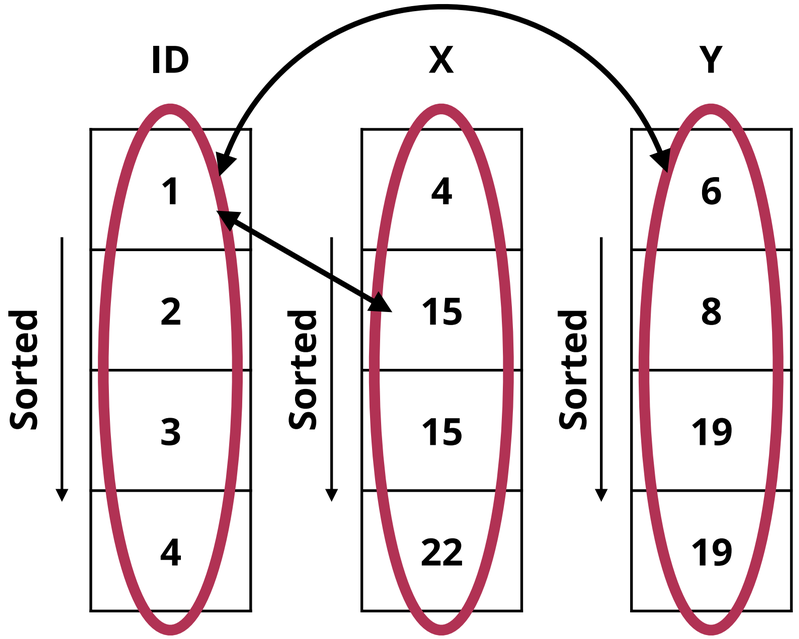
Reverse map allows fast mapping of filtered attributes to their respective IDs

# Sorted multiple dimensional bidirectional map (SMBM)

| ID | X | Y |
|---|---|---|
| 1 | 15 | 6 |
| 2 | 4 | 19 |
| 3 | 22 | 19 |
| 4 | 15 | 8 |

Example table

Stored as




Can **read** the entire data structure in parallel in 1 clock cycle

**Add** and **Delete** can be issued every clock cycle with a latency of 2 cycles

# Add operations

Add (5, 10, 17)

| ID | X | Y |
|---|---|---|
| 1 | 4 | 6 |
| 2 | 15 | 8 |
| 3 | 15 | 19 |
| 4 | 22 | 19 |

| ID | X | Y |
|---|---|---|
| 1 | 4 | 6 |
| 2 | 15 | 8 |
| 3 | 15 | 19 |
| 4 | 22 | 19 |

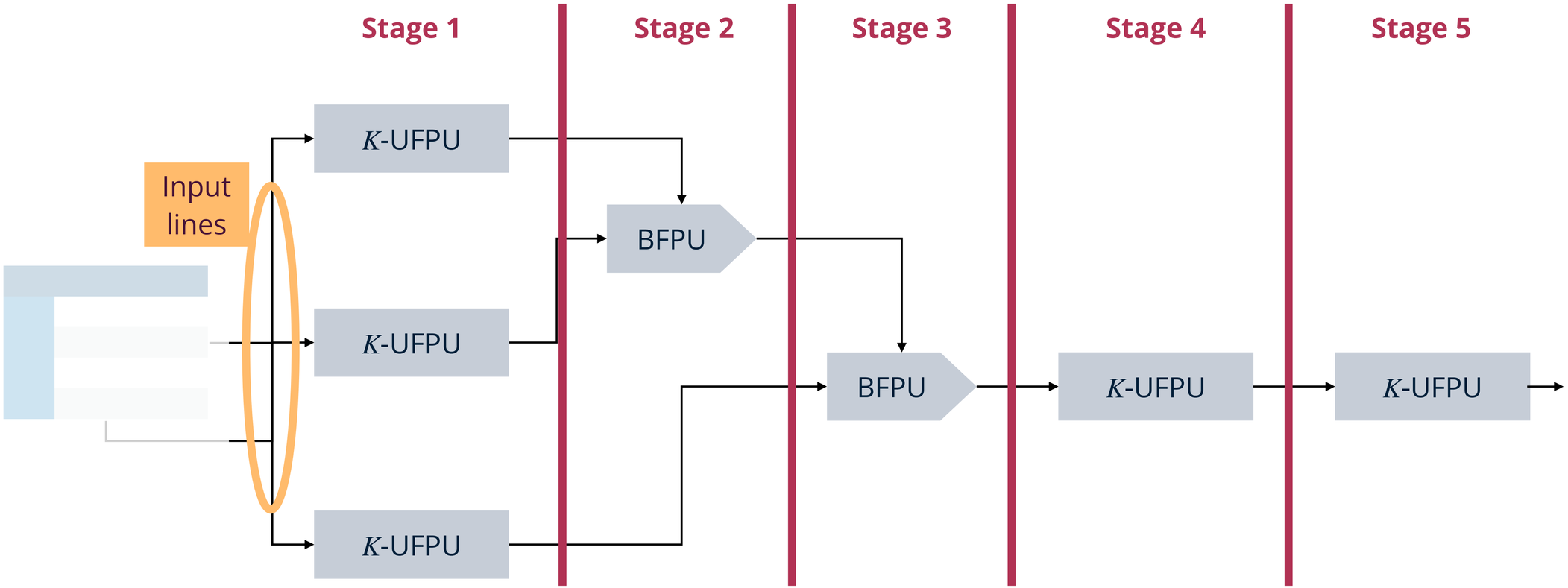
**First clock cycle:** identify the locations for insertion in all lists

| ID | X | Y |
|---|---|---|
| 1 | 4 | 6 |
| 2 | 10 | 8 |
| 3 | 15 | 17 |
| 4 | 15 | 19 |
| 5 | 22 | 19 |

**Second clock cycle:** add the entries, shift the other entries, and update the pointers

Add operation takes two cycles, but the write is **atomic** (only in the second cycle).
**No concurrency issue** for SMBM reads/writes.

# Programmable filter pipeline

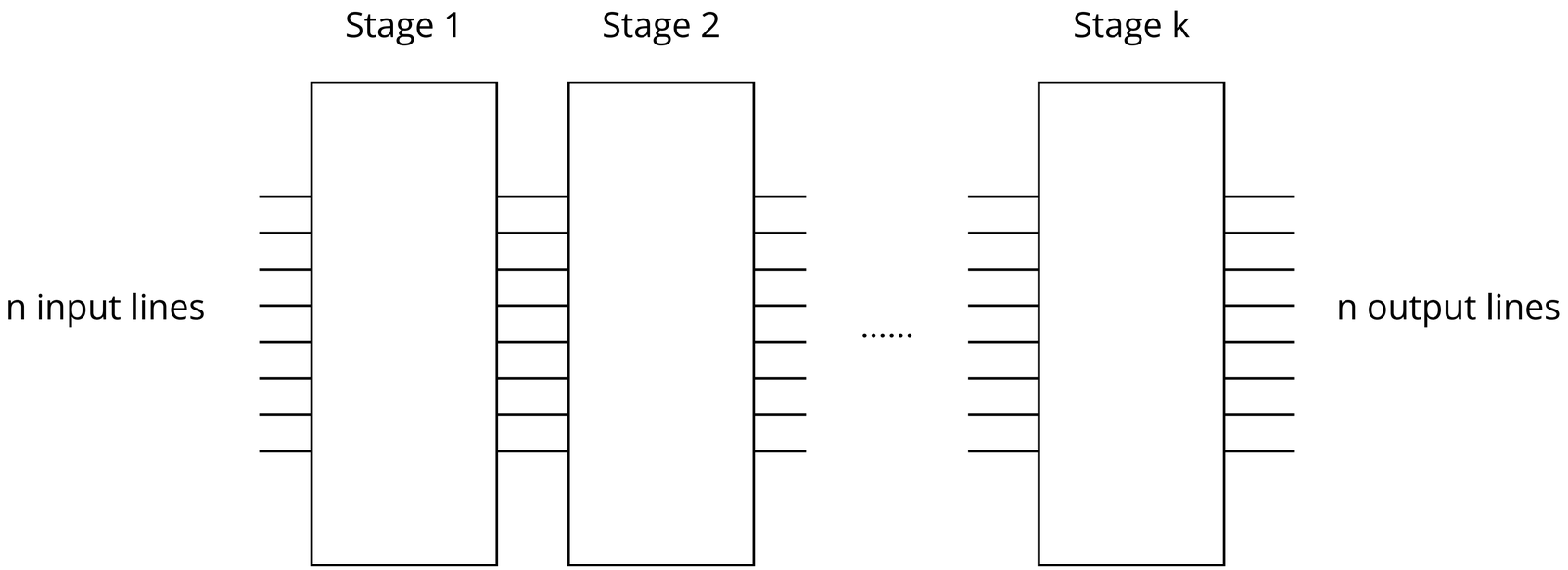



How to design a **fully programmable** and **fast** filter pipeline?

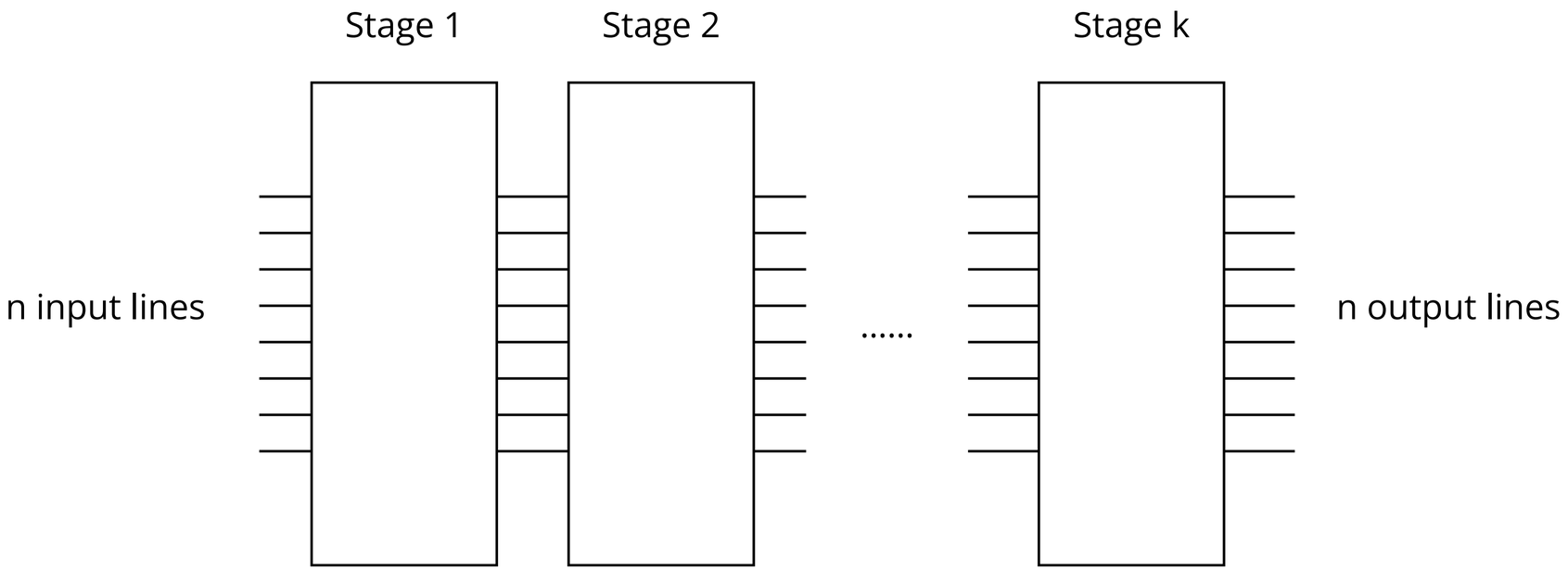
Can express any chain of filter operations (with chain length <=k on the n input lines)

Runs at line rate

# Filter pipeline layout



# Reconfigurable pipeline



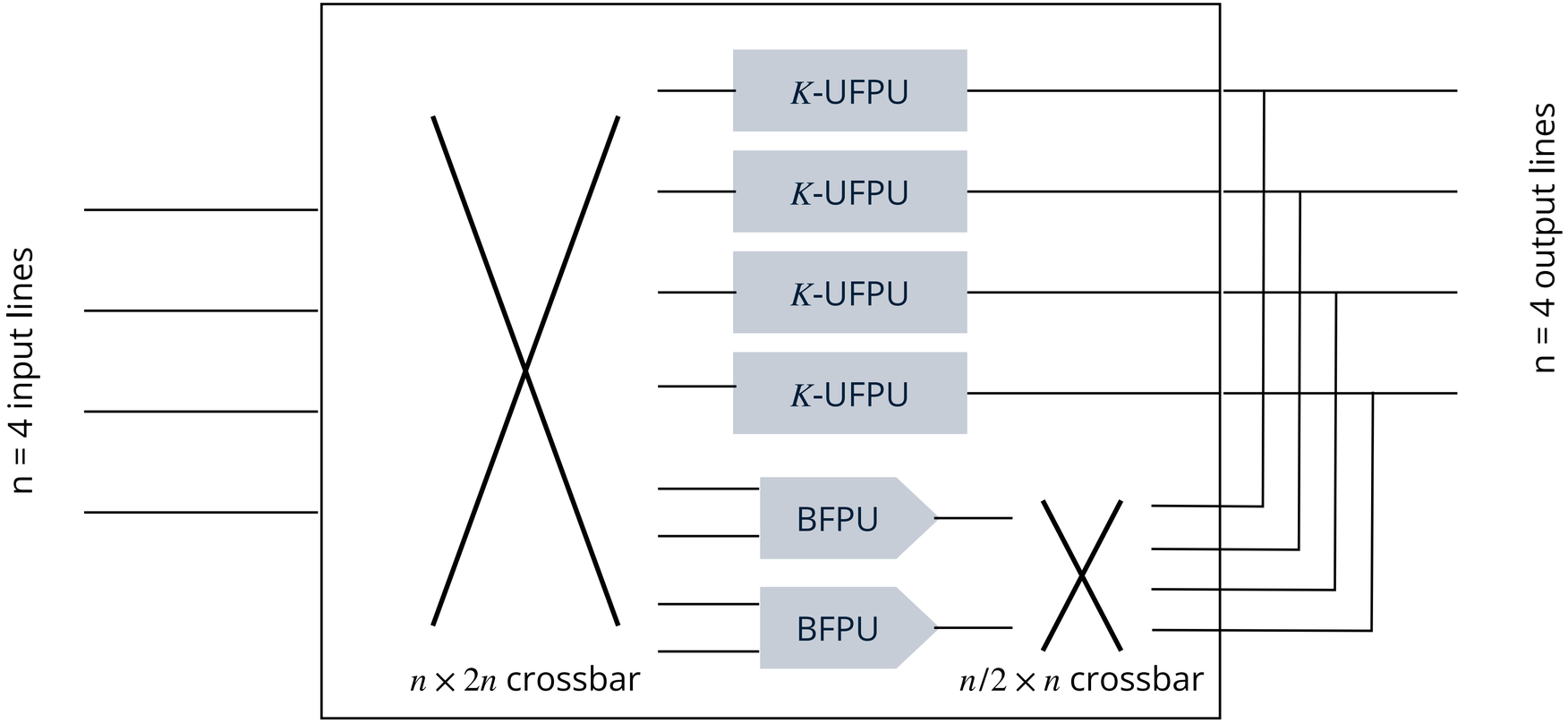

**Sufficient condition for full configurability**

In each stage:

1. Ability to apply **any** filter operation to an input (or pair of input) line
2. Ability to connect the output of a filter operation to **any** output line

# Single pipeline stage design

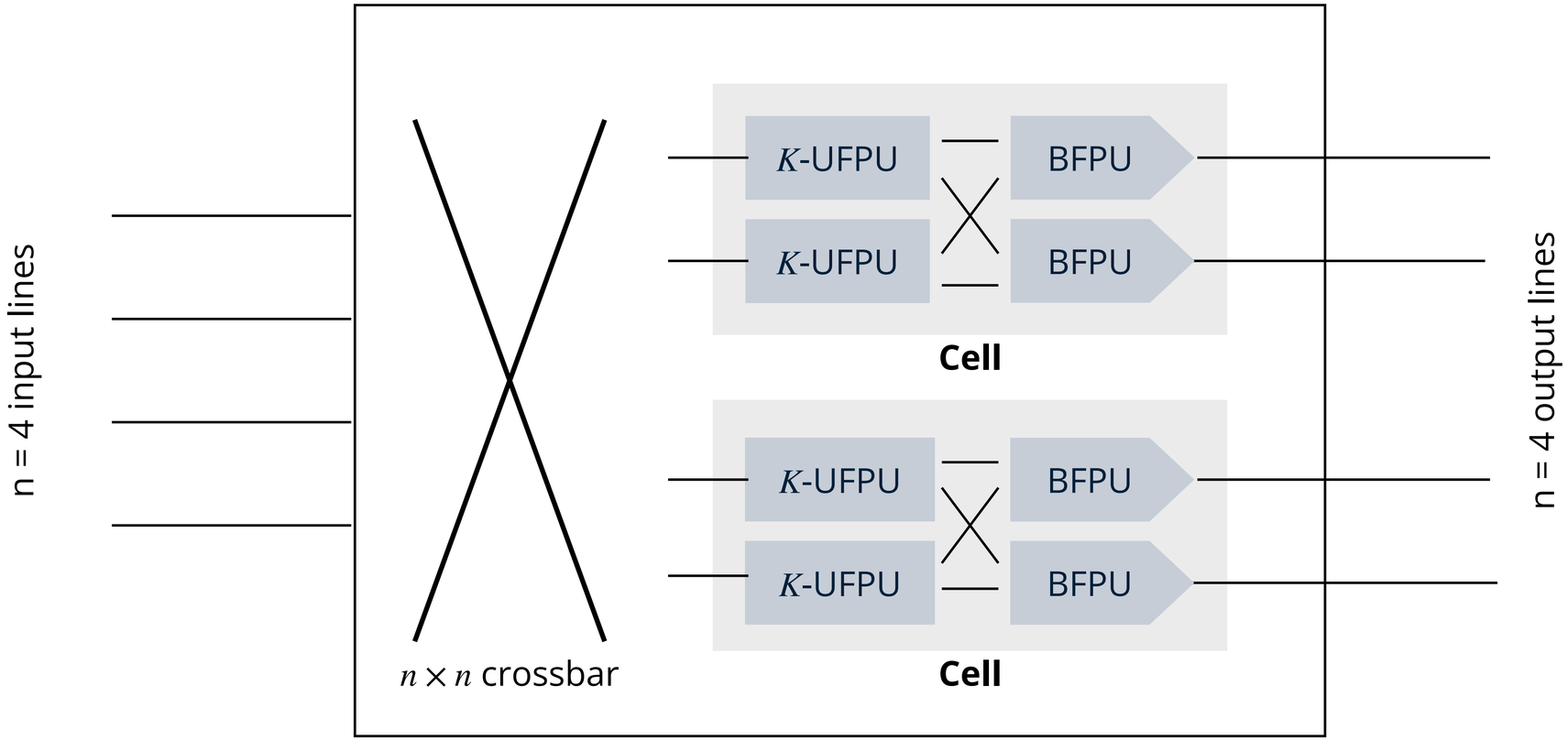
A naive design with both K-UFPUs and BFPUs




High overhead

# Single pipeline stage design

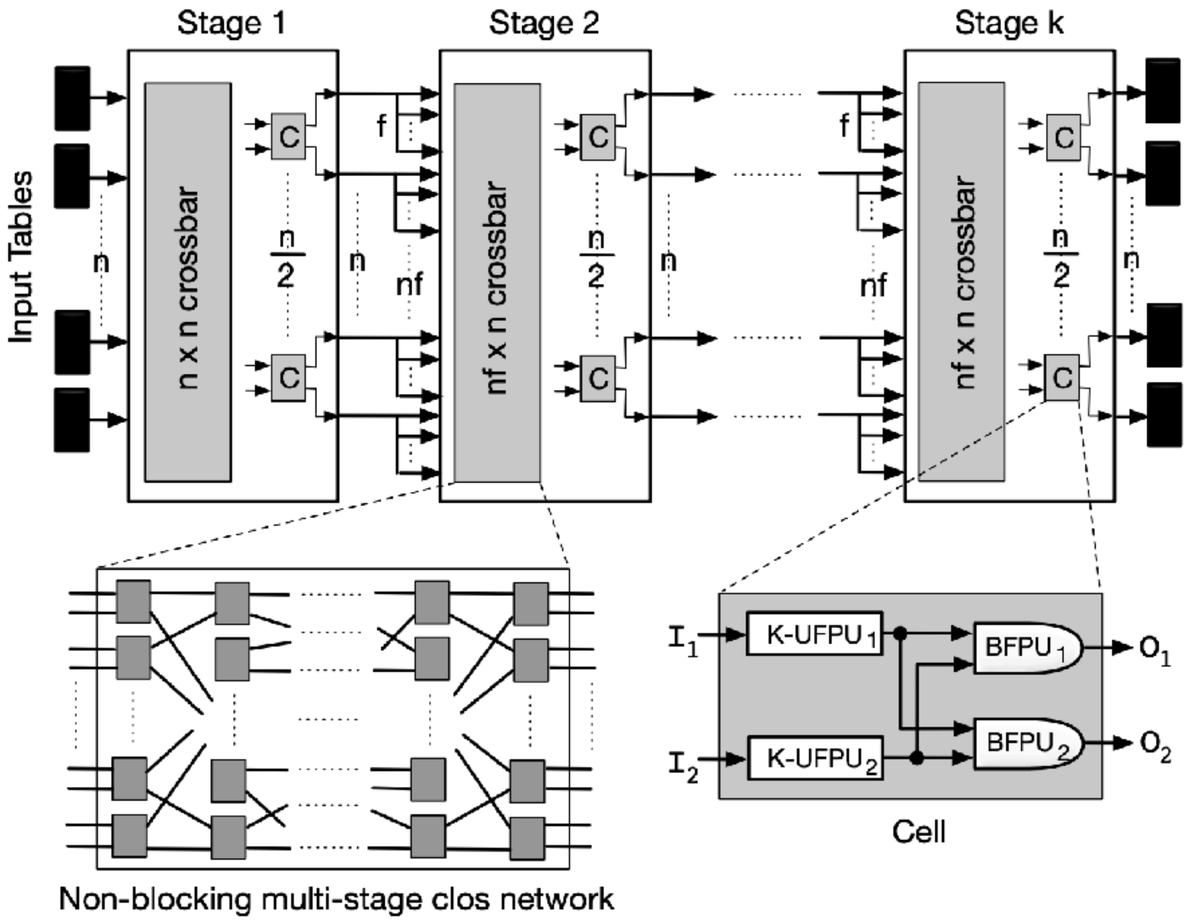
A new design with low overhead




The pipeline is proved to be fully programmable.

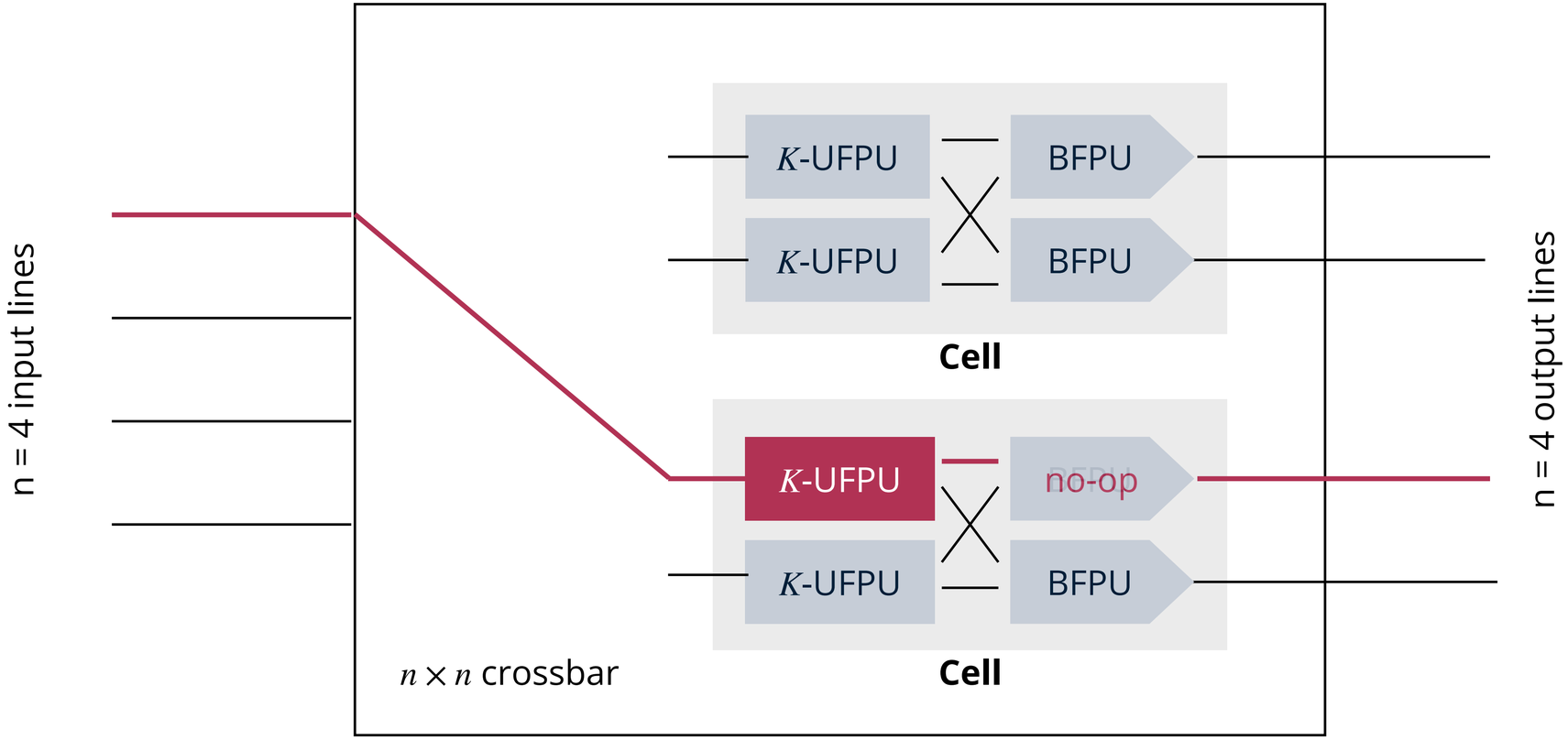
# Pipeline hardware design

$f$ is the **fanout** for each output: the output may be consumed by multiple inputs in the next stage



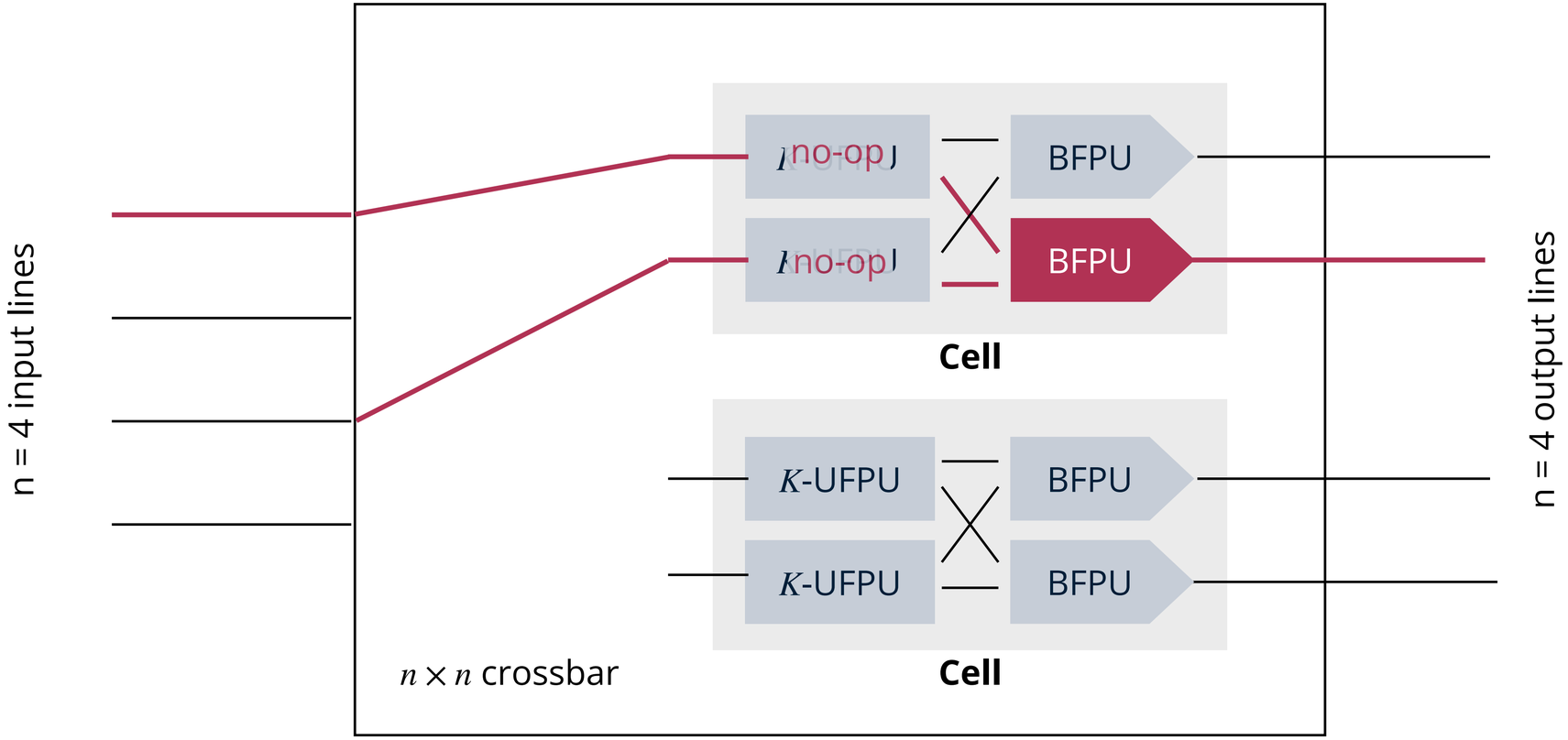
# Example configuration

Apply a UFPU op on input 1 and connect to output 3



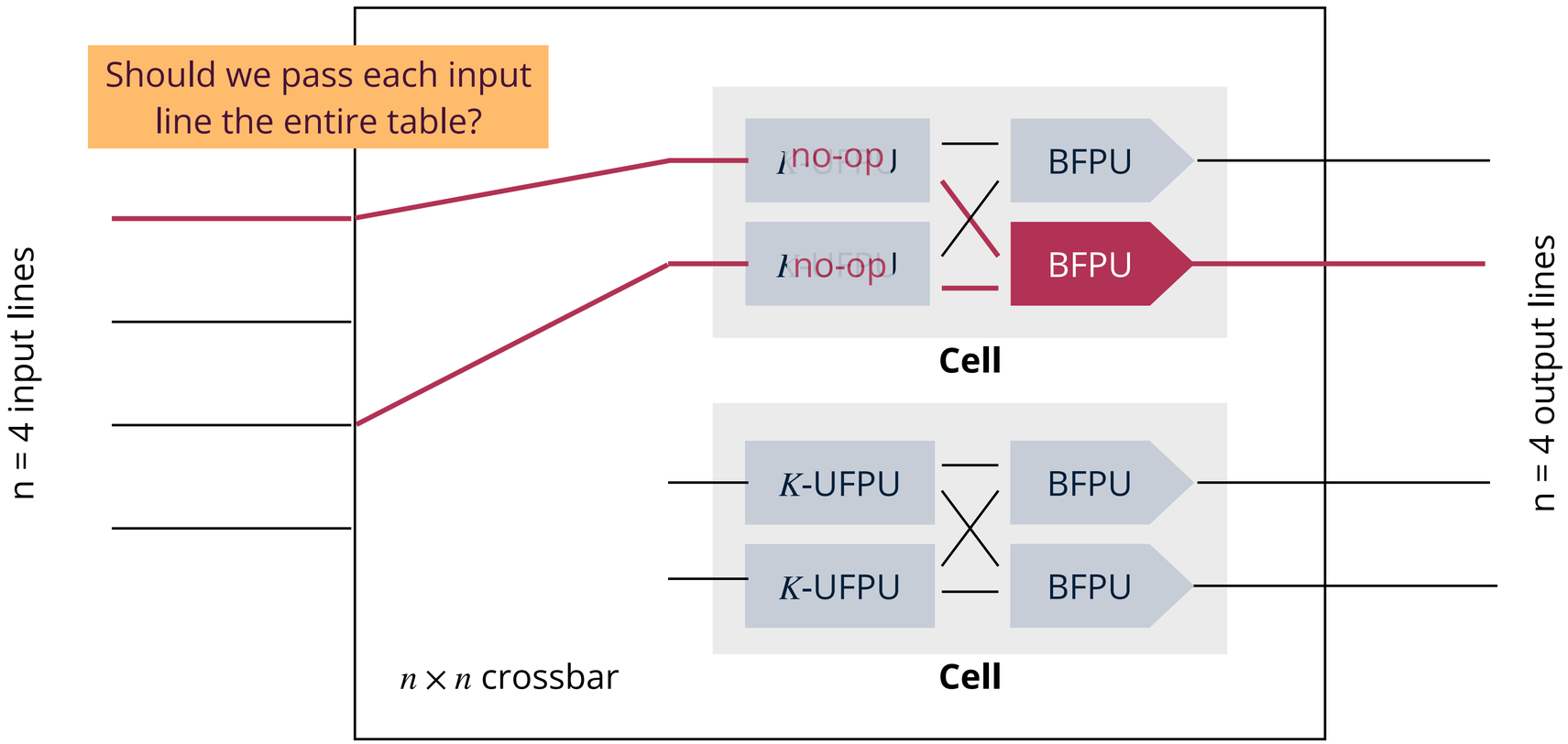
# Example configuration

Apply a UFPU op on input 1 and 3 and connect to output 2

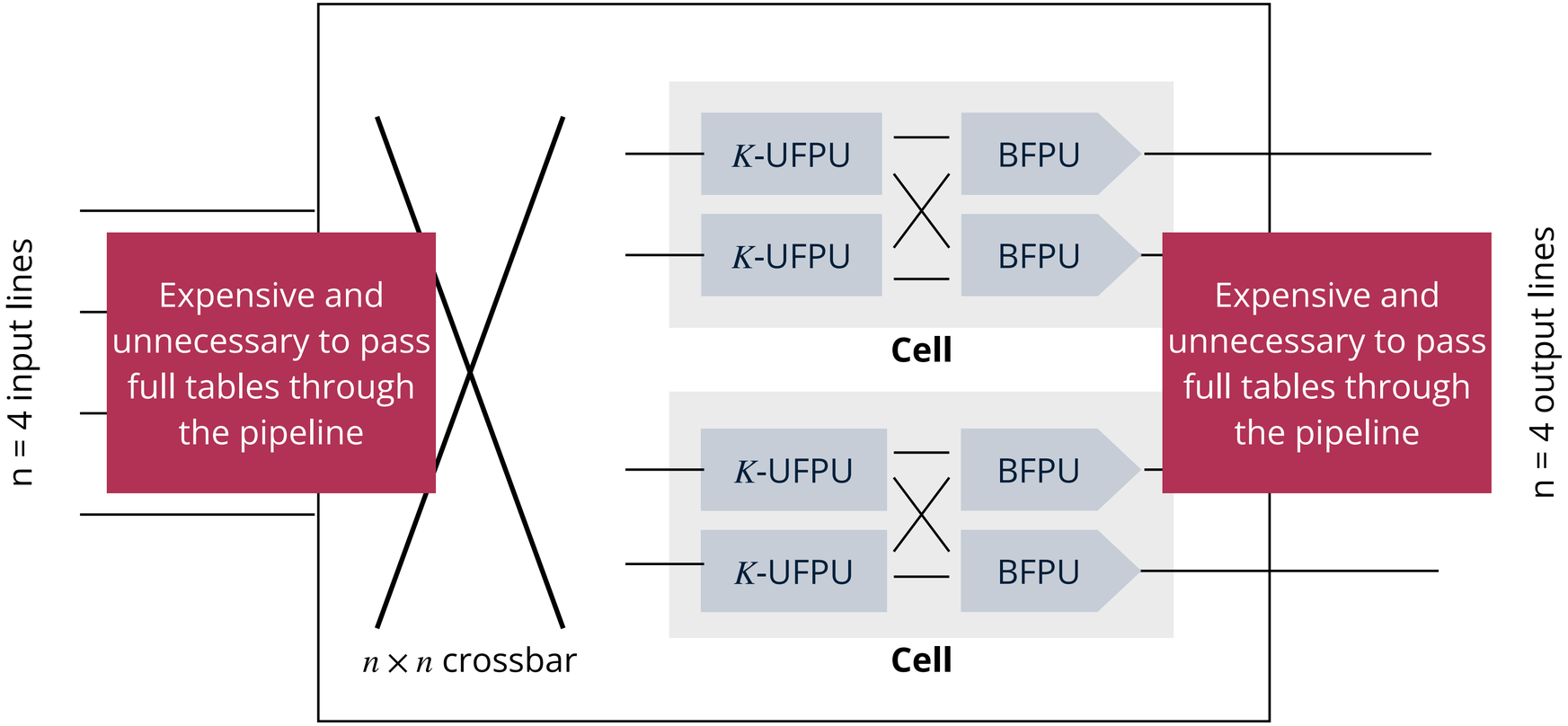


# Example configuration

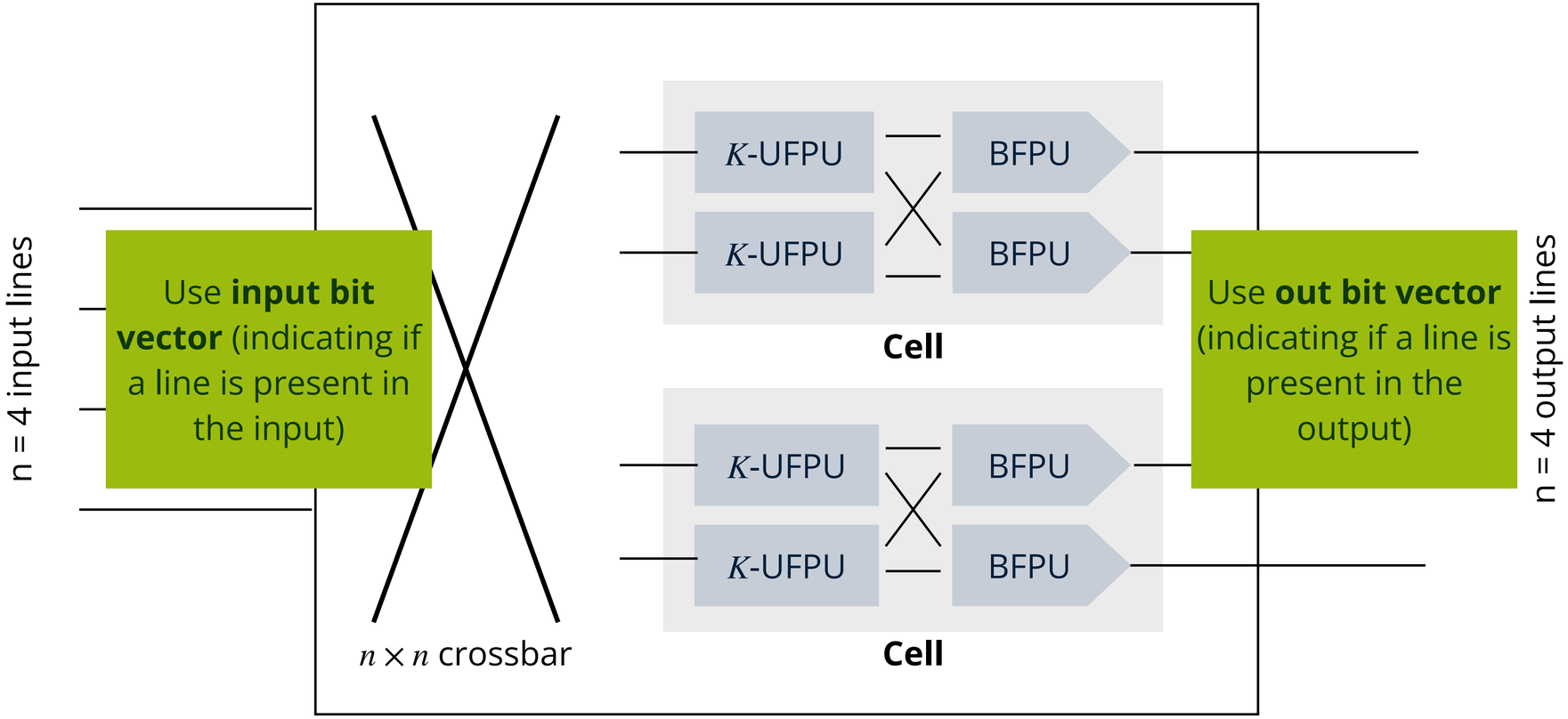
Apply a UFPU op on input 1 and 3 and connect to output 2



# Input lines design



# Input lines design



# UFPU execution example

**Unary filter processing unit**

```
predicate(inp_bit_vector, attrX rel_op val)
```

First clock cycle:

```
temp_list[i] =
if (int_bit_vector[attrX_list[i]→id_ptr])
then attrX_list[i]
else NULL
```

Second clock cycle:

```
out_bit_vector[temp_listp[i]→id_ptr] =
if ((temp_list[i] ≠ NULL) & (temp_list[i]→val rel_op val))
then 1
else 0
```

# BFPU execution example

**Binary filter processing unit**

```
no-op(tab1, tab2, choice)      out_bit_vector = if (choice == 0)
                               then inp_bit_vector_1 else inp_bit_vector_2

        union(tab1, tab2)      inp_bit_vector_1 bitwise-OR inp_bit_vector_2

 intersection(tab1, tab2)      inp_bit_vector_1 bitwise-AND inp_bit_vector_2

   difference(tab1, tab2)      inp_bit_vector_1 bitwise-AND ~inp_bit_vector_2
```

All can be completed in exactly one clock cycle.

# Programmable Multi-Dimensional Table Filters for Line Rate Network Functions

Vishal Shrivastav
Purdue University

## ABSTRACT

The ability to filter entries in the data plane from a set or table of resources (e.g., network paths, servers, switch ports) based on multi-dimensional policies over stateful resource-specific metrics (e.g., filter paths with utilization < 0.6 and latency < 3us) is critical for several key network functions, such as performance-aware routing, resource-aware load balancing, network diagnosis, security and firewall. However, current generation of programmable switches do not support table-wide stateful filtering at line rate. We present **Thanos**, which augments the existing programmable switch pipeline with support for programmable multi-dimensional filtering over a set of resources. Thanos seamlessly integrates with multi-terabit programmable switch pipelines at nominal chip area overhead. Our evaluation, based on an FPGA prototype and a simulator, shows that policies expressed in Thanos can improve the performance of key network functions by up to 1.7× compared to state-of-the-art.

## CCS CONCEPTS

• Networks → Programmable networks; • Hardware → Networking hardware;

## KEYWORDS

Programmable Networks; Switch Architecture

**ACM Reference Format:**
Vishal Shrivastav. 2022. Programmable Multi-Dimensional Table Filters for Line Rate Network Functions. In *ACM SIGCOMM 2022 Conference (SIGCOMM '22), August 22–26, 2022, Amsterdam, Netherlands.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3544216.3544266

port) from a given set of resources, based on certain filtering policy specified over the stateful metrics associated with the resources. Below are some examples.

- **Performance-aware routing [9].** The path to route a packet or flow is chosen based on the current status of the network path metrics. Below is an example.

```
From the set of all paths, select the path
with delay < d and utilization < u
```

**Figure 1: An example routing policy.**

- **Congestion-aware load balancing [2, 8].** The path to load balance a packet or flow or flowlet is chosen based on the current congestion along the path [2, 14] or queuing at egress switch port [8]. Examples include:

```
Randomly choose d out of N possible egress ports.
Find the one with the current minimum queue occupancy
between these d samples and m least loaded samples
from the last time slot, and send packet on that port
```

**Figure 2: Load balancing policy used in DRILL [8].**

```
From the set of all paths, select the path
with least congestion
```

**Figure 3: Load balancing policy used in CONGA [2].**

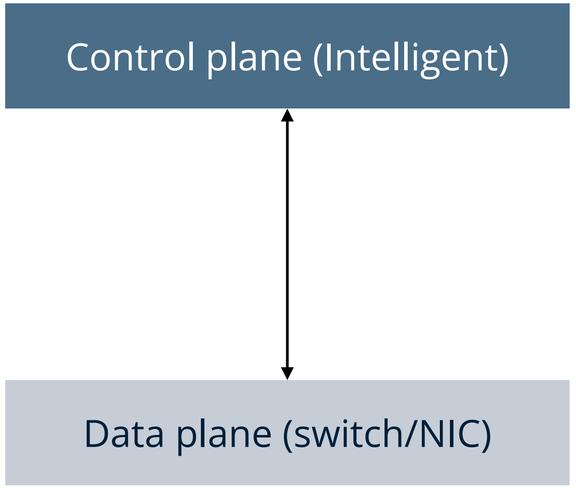
- **Stateful L4 load balancing [18, 22].** The server to map a new L4 flow is chosen based on the resource availability at the load balancing servers. Below is an example.

```
From the set of all servers, select the server
```

SIGCOMM 2022

# How to support machine learning inference on switch data plane?

# Network management



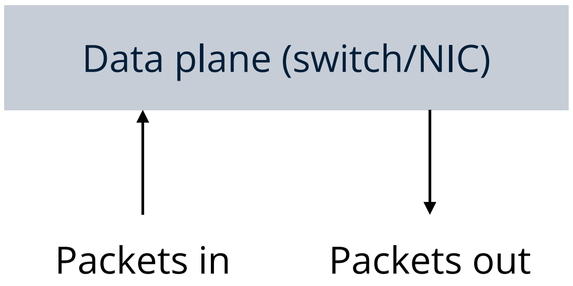

Run complex optimization algorithms
**Slow** but **intelligent**

Apply simply policies (e.g., rule matching)
**Fast** but **dumb**

# Data plane network management

**Examples:**
- Congestion control
- Load balancing (ECMP)
- Queue scheduling

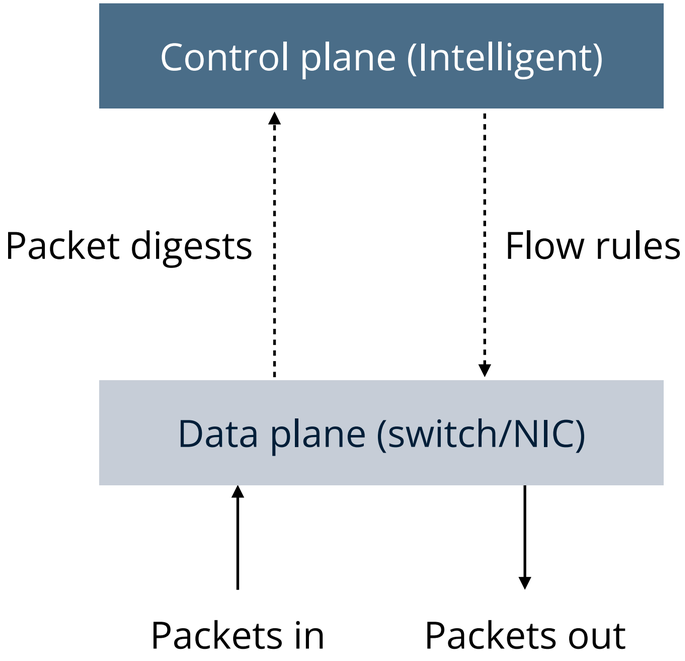
**Characteristics:**
- Operates on packet scale
- Low latency: ns
- High throughput: Tbps
- Use heuristics: hash,...




Apply simply policies (e.g., rule matching)
**Fast** but **dumb**

# Data plane network management

Run complex optimization algorithms
**Slow** but **intelligent**



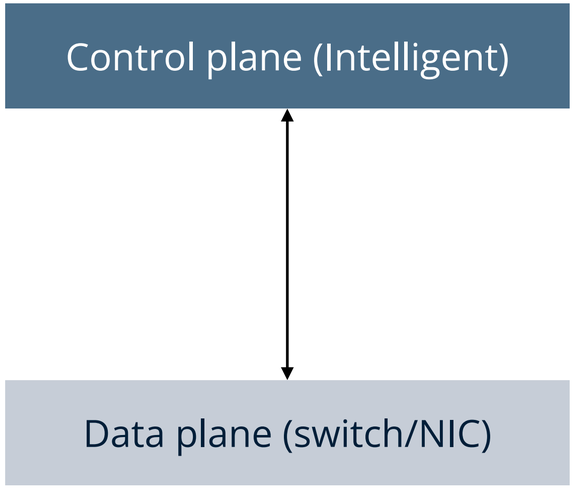

**Examples:**
- Anomaly detection
- Automation
- Recommendation

**Characteristics:**
- Can do machine learning
- Operates on flows
- Latency: ms
- Low throughput

| Accelerator | Latency (ms) |
|---|---|
| Broadwell Xeon | 0.67 |
| Tesla T4 GPU | 1.15 |
| Cloud TPU v2-8 | 3.51 |

Machine learning inference time

# Network management

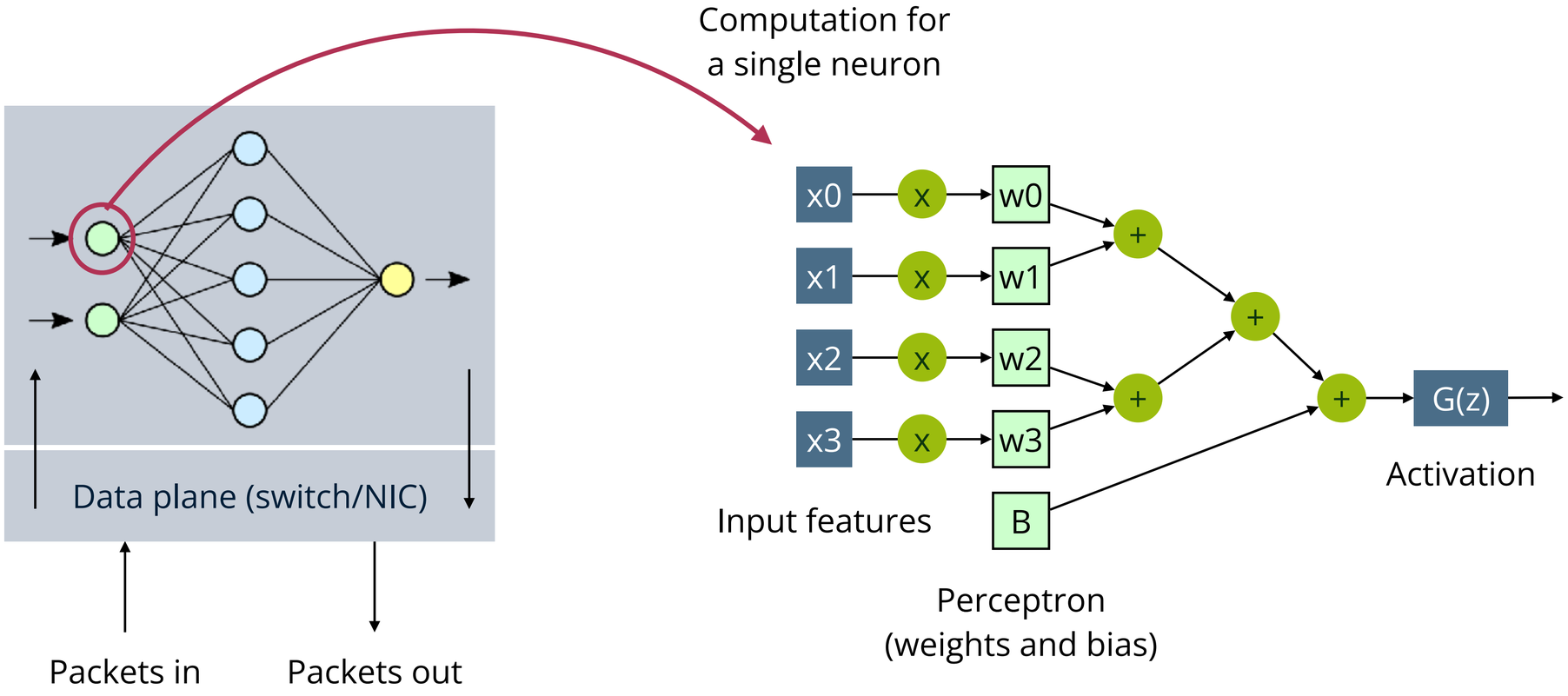



Run complex optimization algorithms
~~**Slow**~~ but **intelligent**

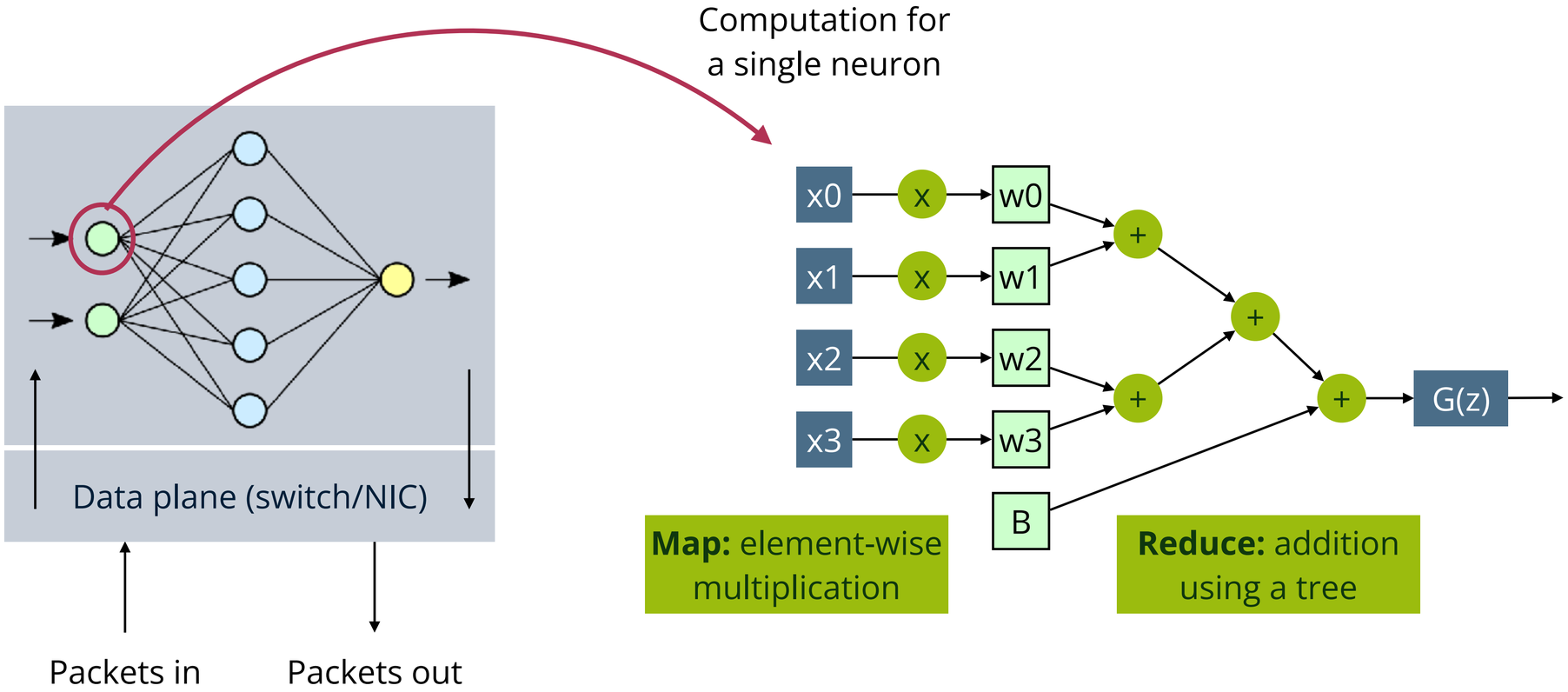
Apply simply policies
**Fast** but ~~**dumb**~~

Can we achieve fast and intelligent network management?

# Use machine learning inference in the data plane

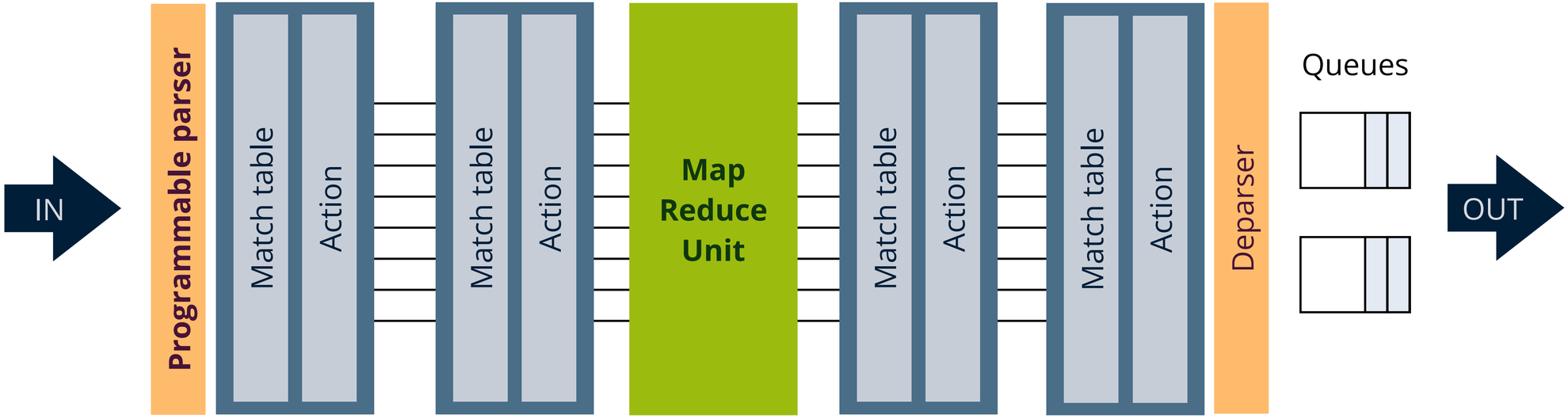


# Computations for machine learning inference



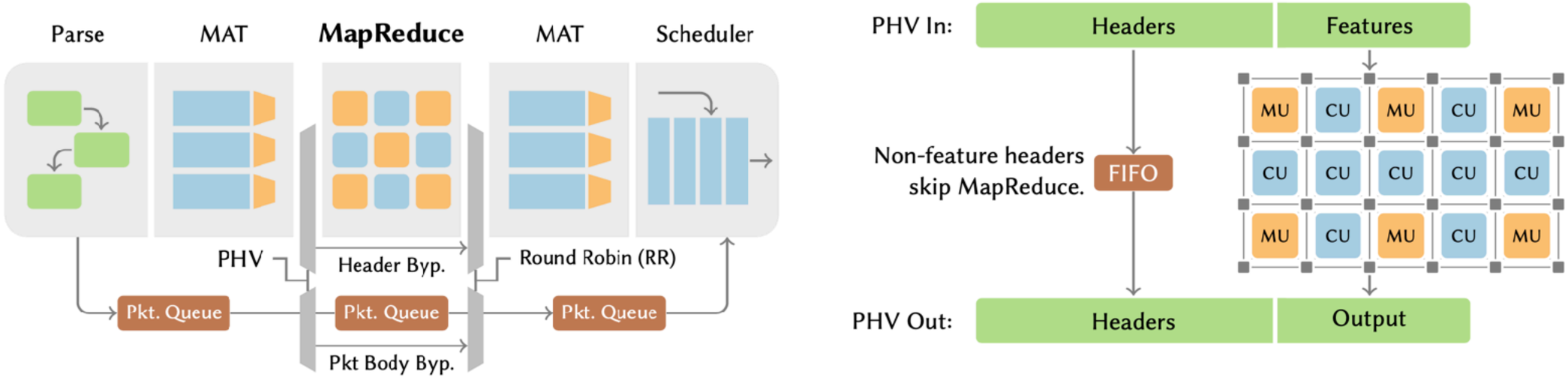

**Possible** for small scales but **inefficient** with the RMT architecture.

# Taurus: programmable Map-Reduce in the data plane

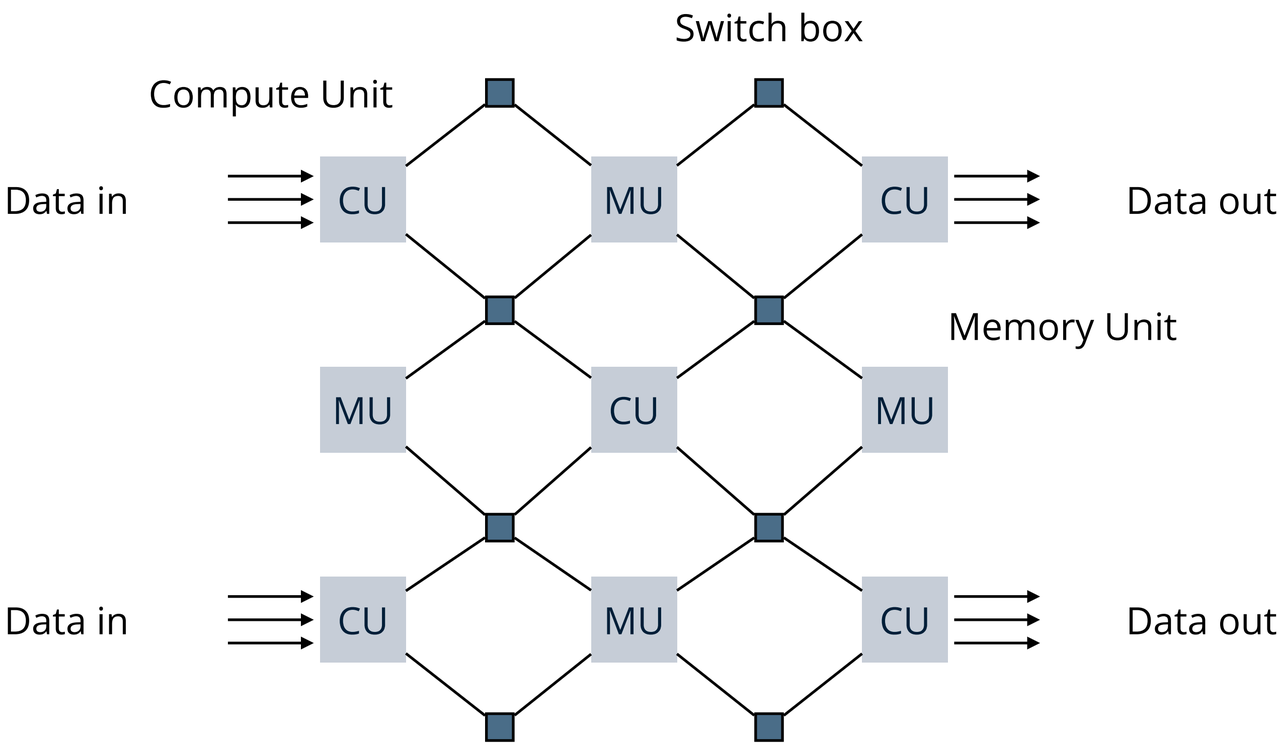



Introducing a domain-specific layer between RMT stages for MapReduce operations.

# Taurus design

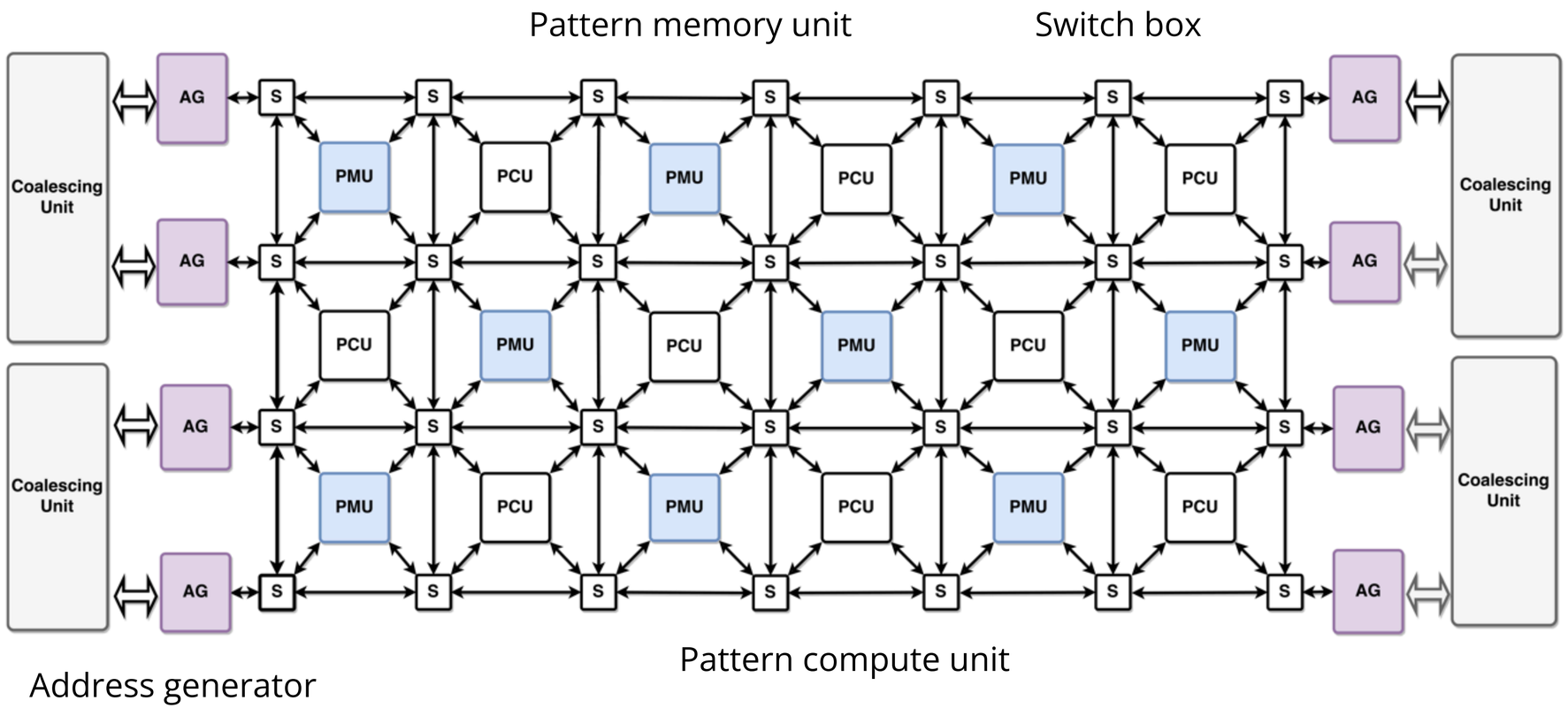


# Building block of Taurus



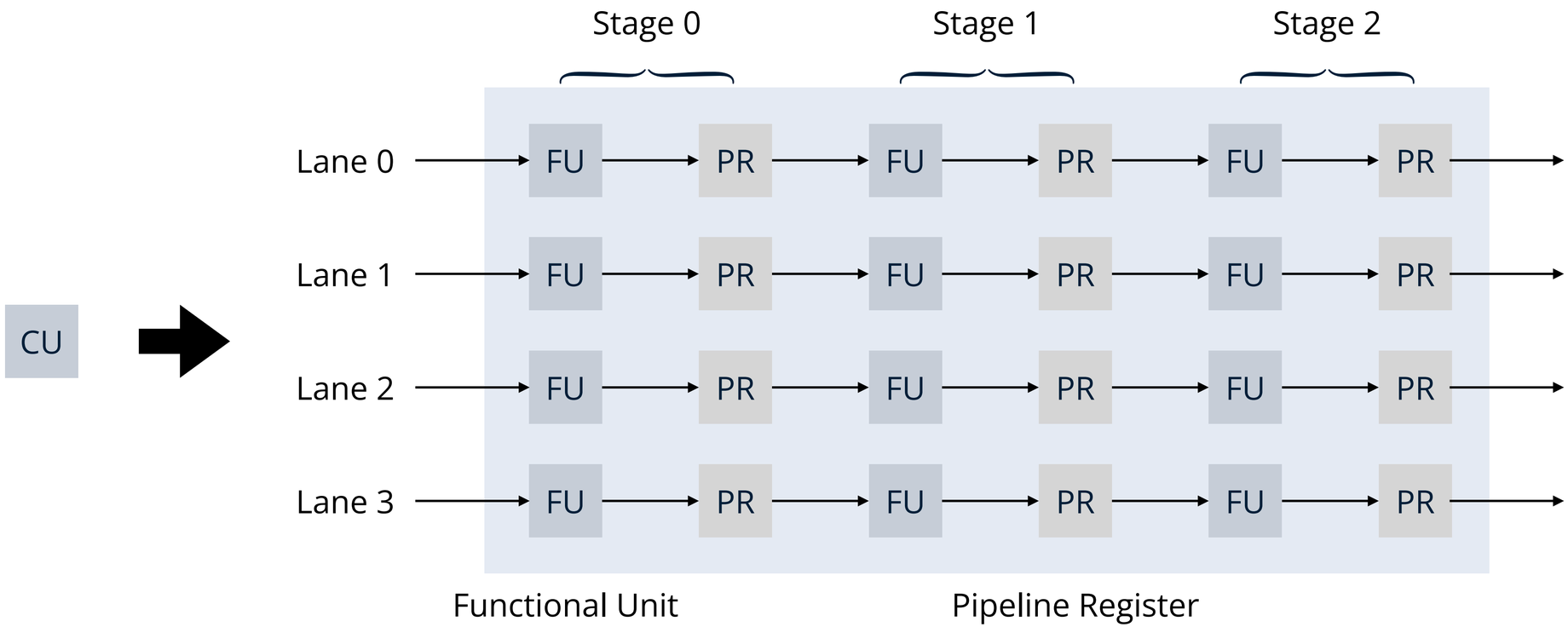

Raghu Prabhakar et al. Plasticine: A Reconfigurable Architecture for Parallel Patterns, ISCA 2017.

# Plasticine: hardware architecture

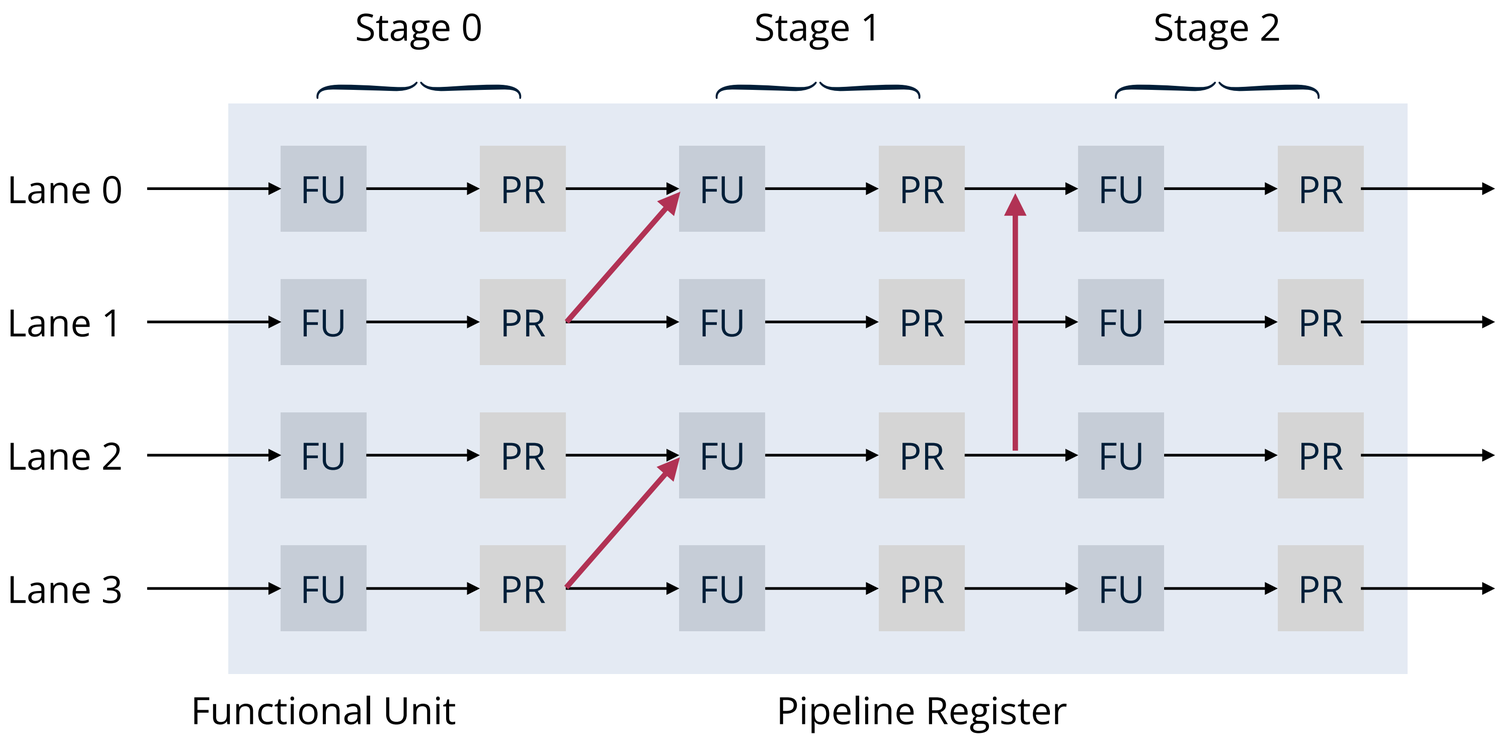



Raghu Prabhakar et al. Plasticine: A Reconfigurable Architecture for Parallel Patterns, ISCA 2017.

# Compute unit design



# Reduction network for condensing vectors to scalars



# MapReduce control block in P4

```
Control Parser (...) {...}
Control PreProcessMAT (...) {...}
Control MapReduce( inout metadata FeatureSet,
                   inout metadata Output ) {
  Weights = loadModelFromFile(Anomaly.model)
  LinearResults = Map(sizeOf(Weights[0])) { i =>
      Mult_Results = Map(sizeOf(Weights[1])) { j =>
          Weights[i,j] * FeatureSet[j] }
      Reduce(Mult_Results) { (x,y) => x + y } }
  Output = Map(sizeOf(LinearResults)) { k =>
      ReLU(LinearResults[k])
} }
Control PostProcessMAT (...) {...}
Control Deparser (...) {...}
```

Iterates over all the neurons in each layer

Apply the activation function

**Map** and **Reduce** constructs in syntax of the Spatial language

David Koeplinger, et al. Spatial: A Language and Compiler for Application Accelerators. ACM/SIGPLAN PLDI 2018.

# What about floating point

Taurus uses 8-bit fixed point numbers for operations

| DNN Kernel | Model Accuracy (%) | | |
|---|---|---|---|
| | float32 | fix8 | Diff. |
| $4 \times 10 \times 2$ | 67.06 | 67.01 | -0.05 |
| $4 \times 5 \times 5 \times 2$ | 67.02 | 66.95 | -0.07 |
| $4 \times 10 \times 10 \times 2$ | 67.04 | 67.02 | -0.02 |

Minimal accuracy loss

| Precision | Area ($\mu m^2$) | Power ($\mu W$) |
|---|---|---|
| fix8 | 670 | 456 |
| fix16 | 1338 | 887 |
| fix32 | 2949 | 2341 |

Significant gains on area and power

**Unlocking the Power of Inline Floating-Point Operations on Programmable Switches**

Yifan Yuan *UIUC* · Omar Alama *KAUST* · Jiawei Fei *KAUST & NUDT* · Jacob Nelson *Microsoft Research* · Dan R. K. Ports *Microsoft Research*
Amedeo Sapio *Intel* · Marco Canini *KAUST* · Nam Sung Kim *UIUC*

**Abstract**

The advent of switches with programmable dataplanes has enabled the rapid development of new network functionality, as well as providing a platform for acceleration of a broad range of application-level functionality. However, existing switch hardware was not designed with application acceleration in mind, and thus applications requiring operations or datatypes not used in traditional network protocols must resort to expensive workarounds. Applications involving floating point data, including distributed training for machine learning and distributed query processing, are key examples.

In this paper, we propose FPISA, a floating point representation designed to work efficiently in programmable switches. We first implement FPISA on an Intel Tofino switch, but find that it has limitations that impact throughput and accuracy. We then propose hardware changes to address these limitations based on the open-source Banzai switch architecture, and synthesize them in a 15-nm standard-cell

been shown to benefit from this in-network computation [94].

However, an important class of applications has struggled to take advantage of in-network computation: those using floating point (FP) values. These occur in two broadly-deployed datacenter applications: distributed training for machine learning, and distributed data processing systems. Since programmable switches were originally optimized for networking applications, their design includes basic support only for integer operations. Applications wanting to take advantage of in-network computation with floating point values have so far worked around this in one of three ways.

The first approach is to approximate floating point operations in software running on end-hosts. This is the approach taken by SwitchML [98] as it sums gradient vectors as part of training deep neural networks. For each chunk of gradient vector elements, SwitchML executes a protocol that requires running code to convert between floating point and integer values on end hosts, as well as performing two rounds of com-

NSDI 2022

# Taurus: A Data Plane Architecture for Per-Packet ML

Tushar Swamy
Stanford University
United States of America

Alexander Rucker
Stanford University
United States of America

Muhammad Shahbaz
Purdue University
United States of America

Ishan Gaur
Stanford University
United States of America

Kunle Olukotun
Stanford University
United States of America

## ABSTRACT

Emerging applications—cloud computing, the internet of things, and augmented/virtual reality—demand responsive, secure, and scalable datacenter networks. These networks currently implement simple, per-packet, data-plane heuristics (e.g., ECMP and sketches) under a slow, millisecond-latency control plane that runs data-driven performance and security policies. However, to meet applications' service-level objectives (SLOs) in a modern data center, networks must bridge the gap between line-rate, per-packet execution and complex decision making.

In this work, we present the design and implementation of *Taurus*, a data plane for line-rate inference. Taurus adds custom hardware based on a flexible, parallel-patterns (MapReduce) abstraction to programmable network devices, such as switches and NICs; this new hardware uses pipelined SIMD parallelism to enable per-packet MapReduce operations (e.g., inference). Our evaluation of a Taurus switch ASIC—supporting several real-world models—shows that Taurus operates orders of magnitude faster than a server-based control plane while increasing area by 3.8% and latency for line-rate ML models by up to 221 ns. Furthermore, our Taurus FPGA prototype achieves full model accuracy and detects two orders of magnitude more events than a state-of-the-art control-plane anomaly-detection system.

## 1 INTRODUCTION

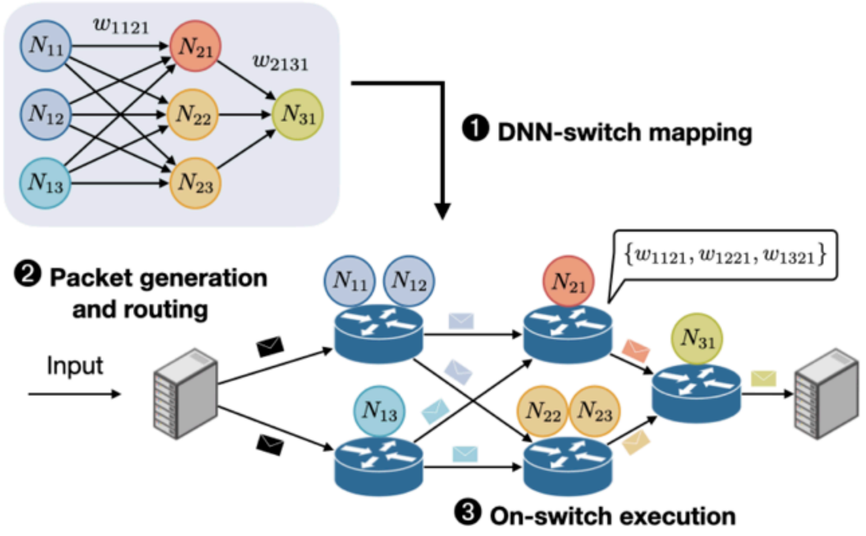
Maintaining strict security and service-level objectives (SLOs) for modern disaggregated workloads (e.g., cloud computing, the internet of things, and augmented/virtual reality) demands that computationally intensive management and control decisions are made on the *current state of the entire datacenter network* (e.g., topology, queue sizes, and link and server loads) and applied *per-packet at line rate* [170]. A delay of even a few microseconds in today's petabit-bisection-bandwidth networks [60] could result in missing millions of anomalous packets [8, 17, 154], saturating switch queues and causing congestion [66, 162, 169], excessive retransmissions due to packet drops [37], or imbalanced traffic and server loads [4, 85]. However, current implementations face a dichotomy: they must choose between per-packet, line-rate execution or computational complexity.

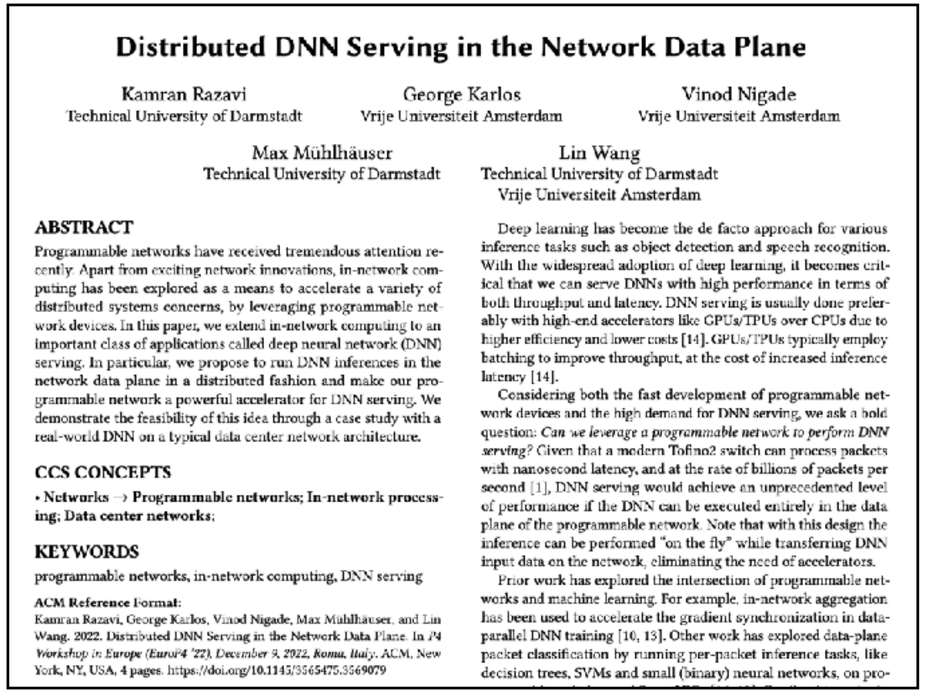
Data-plane ASICs (e.g., switches and NICs) can react in nanoseconds to network conditions, but have a constrained programming model designed to forward packets at line rate (e.g., flow tables [15, 63]). Such a model restricts network operations to simple heuristics [4, 85, 98] or requires having purpose-built tasks in fixed-function hardware (e.g., middleboxes [29, 103]).

Control-plane servers can make complicated, data-driven decisions, but infrequently (typically, on each flow's first packet). The round trip (10 us or more) between the controller and switch fun-

ASPLOS 2022

# Distributed inference in network data plane



## Distributed DNN Serving in the Network Data Plane


Kamran Razavi
Technical University of Darmstadt

George Karlos
Vrije Universiteit Amsterdam

Vinod Nigade
Vrije Universiteit Amsterdam

Max Mühlhäuser
Technical University of Darmstadt

Lin Wang
Technical University of Darmstadt
Vrije Universiteit Amsterdam



**ABSTRACT**

Programmable networks have received tremendous attention recently. Apart from exciting network innovations, in-network computing has been explored as a means to accelerate a variety of distributed systems concerns, by leveraging programmable network devices. In this paper, we extend in-network computing to an important class of applications called deep neural network (DNN) serving. In particular, we propose to run DNN inferences in the network data plane in a distributed fashion and make our programmable network a powerful accelerator for DNN serving. We demonstrate the feasibility of this idea through a case study with a real-world DNN on a typical data center network architecture.



**CCS CONCEPTS**

• **Networks** → **Programmable networks; In-network processing; Data center networks;**

**KEYWORDS**

programmable networks, in-network computing, DNN serving

**ACM Reference Format:**
Kamran Razavi, George Karlos, Vinod Nigade, Max Mühlhäuser, and Lin Wang. 2022. Distributed DNN Serving in the Network Data Plane. In *P4 Workshop in Europe (EuroP4 '22), December 9, 2022, Roma, Italy.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3565475.3569079

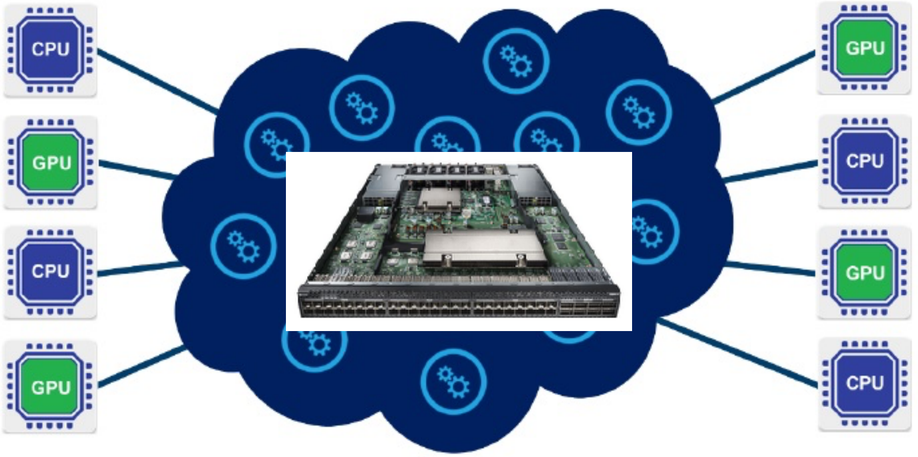

Deep learning has become the de facto approach for various inference tasks such as object detection and speech recognition. With the widespread adoption of deep learning, it becomes critical that we can serve DNNs with high performance in terms of both throughput and latency. DNN serving is usually done preferably with high-end accelerators like GPUs/TPUs over CPUs due to higher efficiency and lower costs [14]. GPUs/TPUs typically employ batching to improve throughput, at the cost of increased inference latency [14].

Considering both the fast development of programmable network devices and the high demand for DNN serving, we ask a bold question: *Can we leverage a programmable network to perform DNN serving?* Given that a modern Tofino2 switch can process packets with nanosecond latency, and at the rate of billions of packets per second [1], DNN serving would achieve an unprecedented level of performance if the DNN can be executed entirely in the data plane of the programmable network. Note that with this design the inference can be performed "on the fly" while transferring DNN input data on the network, eliminating the need of accelerators.

Prior work has explored the intersection of programmable networks and machine learning. For example, in-network aggregation has been used to accelerate the gradient synchronization in data-parallel DNN training [10, 13]. Other work has explored data-plane packet classification by running per-packet inference tasks, like decision trees, SVMs and small (binary) neural networks, on pro-

EuroP4 2022

# Summary




Switch modifications for in-network computing: multi-dimensional table filtering, MapReduce

# Guest lecture

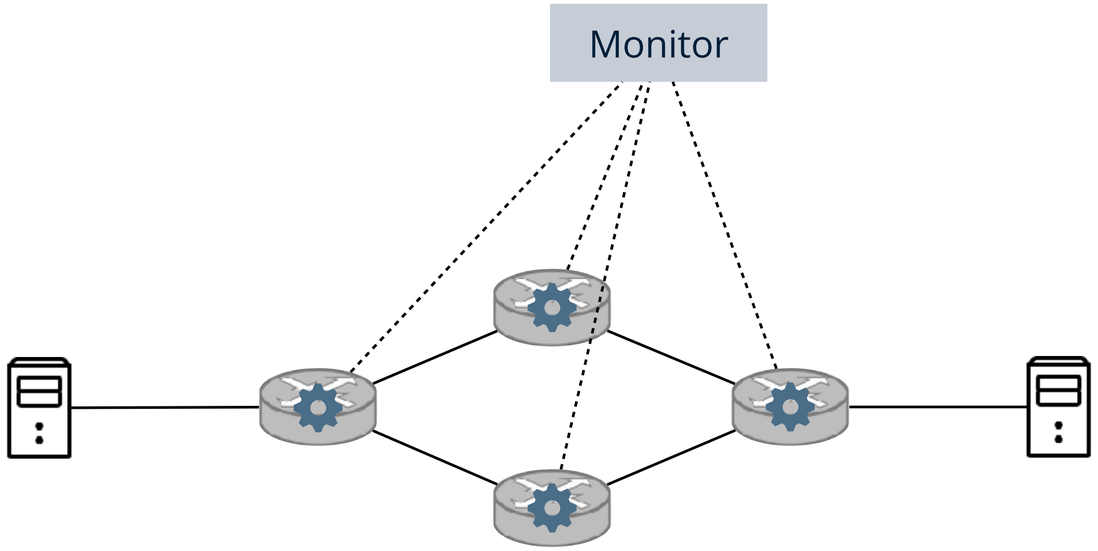
**Title: The next 7000 network switches**

**Fernando Ramos**
Associate Professor
IST, University of Lisbon

When: Wed Dec 14, 11:00
Where: NU-3A06 (and streamed online)

You may find your participation helpful during the exam. :)

# Next time: network monitoring




How to achieve **fast** and **accurate** network monitoring with programmable data plane?